

zVSAM V2 Design and Logic Manual V2.4

Table of Contents

Items marked ### are not yet complete

[Introduction](#)

[Copyright](#)

[Acknowledgements](#)

[Terminology](#)

[Compatibility](#)

[zVSAM compatibility with IBM VSAM](#)

[zVSAM V2 compatibility with zVSAM V1](#)

[API for Assembler and zCOBOL programs](#)

[ACB-based interfaces](#)

[ACB macro](#)

[ACB MACRF keywords](#)

[OPEN macro](#)

[OPEN macro parameters](#)

[OPEN logic](#)

[OPEN execution logic](#)

[Implied OPEN table](#)

[EXLST Macro](#)

[EXLST macro parameters](#)

[Exit logic](#)

[CLOSE macro](#)

[CLOSE macro parameters](#)

[CLOSE logic](#)

[CLOSE execution logic###](#)

RPL-based interfaces

RPL macro

POINT Macro###

GET Macro###

PUT Macro###

ERASE Macro###

CHECK Macro###

ENDREQ Macro###

VERIFY Macro###

GENCB, MODCB, TESTCB and SHOWCB use of the CBMR

CBMR – header

CBMR – body

CBMR – tail

GENCB, MODCB, TESTCB and SHOWCB use of MF=

GENCB, MODCB, TESTCB and SHOWCB parameter types

GENCB BLK=ACB macro

GENCB BLK=EXLST macro

GENCB BLK=RPL macro

MODCB ACB= macro

MODCB EXLST= macro

MODCB RPL= macro

SHOWCB with no specified block type macro

SHOWCB ACB= macro

SHOWCB EXLST= macro

SHOWCB RPL= macro

TESTCB Introduction

TESTCB with no specified block type macro

TESTCB ACB= macro

TESTCB EXLST= macro

TESTCB RPL= macro

Catalog management

Physical structure of the files

Basic Concepts

Files, Blocks, Records

Cluster types and Cluster Components

Record Formats

File Structure

Physical files

Structure of physical files

Block Header Structure

Block Footer Structure

Prefix Block

Prefix Block chain summary

Counters Area and its maintenance

Spacemap Blocks

Record List Structure (RPTR)

Segment Prefix (SPX)

Data Blocks

Data Block Structure (SPANNED=NO)

Data Block Structure (SPANNED=YES)

Data Block

AIX Blocks

AIX Block Structure (Unique)

AIX Block (Unique)

AIX Block Structure (Non-unique)

AIX Block (Non-unique and not segmented)

AIX Block (Non-unique and segmented)

ELIX Block

Index Blocks

Index Block Structure: Single level

Index Block Structure: Two Levels

Index Block Level 0

Index Block other levels

Structure and Functions by dataset type

KSDS Fixed non-Spanned

KSDS Fixed Spanned

KSDS Variable non-Spanned

KSDS Variable Spanned

ESDS Fixed non-Spanned

ESDS Fixed Spanned

ESDS Variable non-Spanned

ESDS Variable Spanned

RRDS Fixed non-Spanned

RRDS Fixed Spanned

RRDS Variable non-Spanned

RRDS Variable Spanned

Logical processes for RPL-based requests

POINT function###

GET function

PUT function

ERASE function

CHECK function###

ENDREQ function###

VERIFY function###

Locking###

Addenda

API for RPL-based interfaces

POINT macro parameters

GET macro parameters

PUT macro parameters

ERASE macro parameters

CHECK macro parameters

ENDREQ macro parameters

VERIFY macro parameters

List of changes

V2.0

V2.1

V2.2

V2.3

V2.4

Introduction

This document describes the structure of the zVSAM component of the z390 assembler and emulator. It consists of the following parts:

[A description of the structure of the interfaces used](#)

[A description of the structure of the files](#)

[A description of the logical processes that implement ACB-based requests](#)

[A description of the logical processes that implement RPL-based requests](#)

[Addenda](#)

Copyright Notice

This document Copyright 2018 – z390 development team

z390 is free software; its associated documentation is equally free. You can redistribute and/or modify both software and documentation under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version

z390 is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with z390; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA.

Acknowledgements

z390's purpose is to provide a source-level compatible assembler, linker and run-time emulation engine for IBM's High-Level Assembler environment. By extension, the zVSAM component described in this document also aims at providing source-level compatibility. As a consequence, all source-level interfaces necessarily mimic the IBM-provided (and IBM-copyrighted) interfaces as described by IBM in publicly accessible documents

The logic and implementation behind these interfaces, however, was developed independently from IBM and is the product of the joint efforts of our team of volunteer developers

Source-level compatibility is a primary goal not only for z390 and zVSAM, but also for other z390 components such as zCOBOL and zCICS.

All IBM publications and software we refer to in this document are copyright IBM Corporation with no exception

The drawings in this document have been made using the draw.io software

As part of the open source for z390 the xml and jpg documents describing these drawings are available with every distribution of z390 that contains this document

Terminology

The reader is assumed to have at least some familiarity with IBM VSAM, to the extent that most of the following acronyms and terms are understood:

-ACB	Access Control Block
-AIX	Alternate IndeX
-CBMR	Control Block Modification Request (zVSAM)
-CI	Control Interval
-ELIX	Extended Level IndeX – extra index level for non-unique AIX (zVSAM)
-ESDS	Entry Sequenced Data Set
-IBM	International Business Machines Corp., USA
-KSDS	Key Sequenced Data Set
-Path	Access to a base cluster, usually through an AIX
-RBA	Relative Byte Address (See note here)
-RDW	Record Descriptor Word in IBM-defined format
-RLF	Record Length Field – (zVSAM)
-RPL	Request Parameter List
-RRDS	Relative Record Data Set
-RRN	Relative Record Number
-SPX	Segment Prefix (zVSAM)
-VSAM	Virtual Storage Access Method
-XRBA	Extended Relative Byte Address (See note here)
-XLRA	Extended Logical Record Address (zVSAM)
-zACB	zVSAM equivalent of the ACB
-zEXLST	zVSAM equivalent of the EXLST
-zRPL	zVSAM equivalent of the RPL

Note:

The XRBA is held in block-relative form, ie. the first record in each block is calculated as
 $BHDRSELF/256 * PFXBLKSZ$

Each subsequent record has its length added to this value according to the RPTR sequence and not the physical sequence (which is reversed). For variable records the length includes the RLF but not any SPX

In this document we also use the following terms. The ones that are used by IBM as well, are intended to have the same meaning they do in IBM manuals

Area	a section of storage with a defined layout, depending on the type of Area
Block	zVSAM equivalent of a Control Interval
Cluster	a set of files that logically belong together
Component	either a data component or an index component of a cluster
Element	a primary key or XRBA in an AIX record
File	a single file as seen by the hosting operating system
Foxes	a value consisting of all high-values i.e. a value of all X'FF' bytes.
List	a structure holding items that are linked together by s
Segment	a portion of a record in a spanned dataset
Segmented	a record that has been split into segments in a spanned dataset
Spanned	an attribute of a dataset that allows records to be split into segments
Sphere	a cluster and all associated AIXs
Table	a structure holding items that are physically adjacent

Compatibility

As this document relates to zVSAM V2, there are two type of compatibility we need to consider. On the one hand we have designed zVSAM to be compatible with IBM VSAM. And on the other hand we need to consider compatibility with z390's zVSAM V1 – the prior implementation of zVSAM in the z390 environment

zVSAM compatibility with IBM VSAM

Our z390 implementation of zVSAM V2 is intended to be source-level compatible with IBM VSAM. This has the following consequences:

1. IBM VSAM documentation with regards to macros and interfaces applies to zVSAM with the exception of parameters and options not supported by zVSAM. Where zVSAM differs in behaviour this is noted in this document. Please refer to the macro descriptions for details
2. Control Blocks (such as ACB, RPL and some others) are not compatible. zVSAM has its own structures. A side effect of this may be that a program's assembled object code may be different in size than on your IBM operating system. On rare occasions you may need an additional base register when porting your program either way
3. As a rule of thumb, a program using VSAM can be ported to z390 and should be able to assemble, link and run without modification – provided it uses only the VSAM features and options that zVSAM supports. And provided the program does not run out of addressability due to different control block lengths

zVSAM V2 compatibility with zVSAM V1

The user of z390's zVSAM component should be aware that zVSAM V2 as described in this document is not compatible with the pre-existing zVSAM V1. We – the development team – apologize for the inconvenience this may cause

We have taken the following measures to facilitate the transition from zVSAM V1 to zVSAM V2:

- 1) We have introduced a new z390 option: ZVSAM which indicates which version of zVSAM you want z390 to use. It takes the following forms:
 - ZVSAM(0) – zVSAM usage is disallowed
 - ZVSAM(1) – zVSAM V1 is enabled, zVSAM V2 is disabled
 - ZVSAM(2) – zVSAM V2 is enabled, zVSAM V1 is disabledFor maximum compatibility the default is set to ZVSAM(1).
The default will be changed to ZVSAM(2) in a future release of z390
- 2) To convert your zVSAM V1 clusters to zVSAM V2 you'll have to take the following steps:
 - unload the existing data from their clusters using REPRO
 - reload your data from your unload files, using ZREPROFor details on how to use REPRO, please refer to the ["z390 VSAM User Guide"](#)
For details on how to use zREPRO, please refer to the ["z390 zVSAM zREPRO User Guide"](#)
- 3) For zVSAM V1 and zVSAM V2 there are distinct macro libraries, MACVSAM1 and MACVSAM2
To use the correct zVSAM maclib, specify the correct version in your maclib concatenation
- 4) If a program is to run with ZVSAM(2), then all submodules that contain an OPEN macro must be re-assembled using MACVSAM2, even those that only use QSAM

API for Assembler and zCOBOL programs

There is much more syntax checking in zVSAM V2 than in IBM macros and this may result in unexpected MNOTES. All macros will continue processing after an MNOTE except for the most serious. This may result in assembler errors. Once the reason for the MNOTE has been resolved the assembler errors will be eliminated.

ACB-based interfaces

The ACB is the primary interface for operations at the cluster level. Each cluster is represented by an ACB.

The ACB interface consists of an ACB control block, possibly an Exit list Control Block, and a set of macros to manage and manipulate the ACB and EXLST control blocks. These macros can be used in your assembler programs. For zCOBOL and/or other higher-level languages, these macros will be generated from specifications for the files as appropriate in the host language's syntax.

The following macros are provided for assembler programs:

- ACB
- ACBD
- CBMR
- GENCB BLK=ACB
- MODCB ACB=
- SHOWCB ACB=
- TESTCB ACB=

Note: The ACB macro defines a statically allocated ACB. This macro is primarily intended for use in non-re-entrant programs. GENCB BLK=ACB should be used to create an ACB in dynamically acquired storage, or in private static storage. MODCB ACB= can be used to modify an existing ACB, whereas SHOWCB ACB= can be used to query specific fields of an ACB and TESTCB ACB= can be used to validate specific fields of an ACB.

- EXLST
- EXLSTD
- GENCB BLK=EXLST
- MODCB EXLST=
- SHOWCB EXLST=
- TESTCB EXLST=

Note: The EXLST macro defines a statically allocated EXLST. This macro is primarily intended for use in non-re-entrant programs. GENCB BLK=EXLST should be used to create an EXLST in dynamically acquired storage, or in private static storage. MODCB EXLST= can be used to modify an existing EXLST, whereas SHOWCB EXLST= can be used to query specific fields of an EXLST and TESTCB EXLST= can be used to validate specific fields of an EXLST.

- OPEN
- CLOSE

Note: OPEN and CLOSE macros can be used to open and close either sequential files represented by a DCB and/or zVSAM files represented by an ACB.

A description of these interfaces as implemented for z390 and zVSAM is detailed in the next chapters.

ACB Macro

The ACB macro will generate an ACB and initialize it according to the parameters specified on the macro invocation

Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB=, TESTCB ACB= and/or MODCB ACB= to inspect, test, and/or modify the ACB's content

All keywords on the ACB macro are optional. Before the cluster is opened, all ACB values can be modified using MODCB ACB=, or by changing the ACB directly. The latter is not recommended, as it is not guaranteed to be portable or compatible with future versions of zVSAM

The table below shows how the ACB macro can be coded

Opcode	Operand	Remarks
[label] ACB	[AM=VSAM]	Designates this ACB as a zVSAM ACB
	[DDNAME=ddname]	DDNAME: name of an environment variable in the host OS holding the name of the cluster to be processed See notes here
	[PASSWD=address]	Address of password for the cluster Points to a single byte length followed by the password eg. X'05',C'ABCDE'
	[EXLST=address]	Address of an exit list. Please see the EXLST macro description for details here
	[MACRF=(keyword list)]	List of keywords for processing options See table below for valid keywords here
	[BUFSP=value]	Max amount of storage (in bytes) to use for buffers See notes here
	[BUFND=value]	Number of data buffers to allocate for this ACB Specify a number between 1 and 65535
	[BUFNI=value]	Number of index buffers to allocate for this ACB Specify a number between 1 and 65535
	[RMODE31=keyword]	Indicates whether buffers and/or control blocks can be allocated above the line See notes here
	[STRNO=1]	Number of concurrent requests allowable for this ACB Specify a number between 1 and 255
	[BSTRNO=value]	Beginning number of concurrent requests allocated to this ACB when a path is opened. Only applies if MACRF=NSR Specify a number between 0 and 255
	[MAREA=address]	Not supported – future option Keyword is flagged as ignored with a warning message
	[MLEN=value]	Not supported – future option Keyword is flagged as ignored with a warning message
	[RLSREAD=keyword]	Not supported – future option Keyword is flagged as ignored with a warning message
	[SHRPOOL=value]	LSR shared pool number – future option

ACB MACRF keywords

Keyword subset	Keyword	Remarks
[ADR <u>KEY</u>]	ADR	Addressed access to ESDS by (X)RBA Using (X)RBA to access a KSDS is not supported
	KEY	Keyed access to a KSDS RRN access to an RRDS
	CNV	Not supported. Keyword is flagged with a warning message
[DFR <u>NDF</u>]	DFR	Allow writes to be deferred
	NDF	Do not defer writes
[DIR <u>SEQ</u> <u>SKP</u>	DIR	Direct access to ESDS, KSDS or RRDS
	SEQ	Sequential access to ESDS, KSDS or RRDS
	SKP	Skip sequential access to KSDS or RRDS Only for keyed access. Allows the use of POINT
[<u>IN</u> <u>OUT</u>]	IN	Read only access for ESDS, KSDS or RRDS
	OUT	Both read and write access for ESDS, KSDS or RRDS
[<u>NIS</u> <u>SIS</u>]	NIS	Normal Insert Strategy for KSDS
	SIS	Sequential Insert Strategy for KSDS
[<u>NRM</u> <u>AIX</u>]	NRM	DDNAME indicates cluster to be processed
	AIX	DDNAME of a path to access an AIX directly, rather than using it to access records in the underlying base cluster
[<u>NRS</u> <u>RST</u>]		Not supported. Keyword is flagged with a warning message
[<u>LSR</u> <u>GSR</u> <u>NSR</u> <u>RLS</u>]		Local, Global or no Shared Buffers. RLS is not supported
[NUB/UBF]		Not supported. Keyword is flagged with a warning message
[CFX/NFX]		Not supported. Keyword is flagged with a warning message
[DDN/DSN]		Not supported. Keyword is flagged with a warning message
[ICI/NCI]		Not supported. Keyword is flagged with a warning message
[LEW/NLW]		Not supported. Keyword is flagged with a warning message

With the exception of the DDNAME parameter explained below, all supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual

DDNAME= notes

DDNAME is required before open is executed. If DDNAME is not supplied on the ACB macro, the label used on the ACB macro is used as DDNAME. If neither is specified, a proper value must be supplied by using MODCB ACB=

In zVSAM the DDNAME refers to the name of an environment variable in the host OS. This variable in turn should contain the path and qualified filename of the cluster to be opened. The qualifier is the name of an environment variable in the host OS and is the path to the assembled catalog

For more information on zVSAM catalogs, please refer to the ["z390 zVSAM Catalog User Guide"](#)

For more information on environment variables, please refer to the ["z390 zVSAM zREPRO User Guide"](#)

BUFSP= notes

Maximum buffer space in virtual storage for this cluster

This is the combined size in bytes of all buffers allocated for this cluster. If $(BUFND + BUFNI) * \text{Block_size}$ exceeds the value specified for BUFSP, then BUFND and BUFNI will be reduced proportionally to keep the total allocation below the limit specified in the BUFSP parameter

RMODE31= notes

Specifies whether buffers and/or control blocks should be allocated below or above the 16M line:

NONE Control Blocks and buffers below 16M

CB Control Blocks above or below 16M, buffers below 16M

BUFF Control Blocks below 16M, buffers above or below 16M

ALL Control Blocks and buffers above 16M or below 16M

The default for RMODE31 is NONE

OPEN macro

A cluster needs to be opened before it can be processed. The open macro is used to open one or more clusters and/or one or more sequential files in a single call

Opcode	Operand	Remarks
[label] OPEN	(entry[,entry]...)	Each cluster or file requires an entry of two parameters
	[MODE= <u>24</u> /31]	Residency mode of all control blocks involved. Specify 31 if any resides above the line
Entry format:	address,(options)	Address of ACB or DCB, followed by a list of options (for DCB only). For ACB omit the list of options.
	[MF=I or omitted]	Use standard form of OPEN
	[MF=L]	Use list form of OPEN
	[MF=(E,address)]	Use execute form of OPEN

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual

OPEN macro parameters

entry The OPEN macro accepts a list of entries. Each entry consists of two consecutive parameters: an address and an optional list of options

address The address can be specified as an A-type address or as a register. If a register is coded the register number or name must be enclosed in parentheses. The address can be either the address of a DCB or the address of an ACB

options For a DCB options may be encoded according to the [z390 File Access Method Guide](#)
For an ACB the options list is ignored and should be coded as an omitted parameter
Any options (e.g. IN/OUT) are taken from the ACB, not the open parmlist

MF=I or omitted

An open parmlist is generated inline, plus a call to the OPEN SVC using the parmlist

MF=L An open parmlist is generated inline

MF=(E,address) Code to modify/populate the open parameter list at the indicated address, which may be a relocatable constant or a (register), plus a call to the OPEN SVC using the parmlist

Note: IBMs OPEN allows an MF=L to be built which can then be overwritten with an MF=E containing more entries than were originally created, thus causing a storage violation.
zVSAM V2 will return R15=8 if this is attempted

IBMs OPEN allows an MF=E to modify an MF=L with a different mode causing damage to the list
zVSAM V2 will return R15=8 if this is attempted

OPEN logic

Open logic has two major components: the open macro and the actual run-time logic to execute a request to open a file or a number of files

Open parameter list entries have two different formats depending on the MODE parameter.

When MODE=24 then each entry is one fullword

When MODE=31 then each entry is two fullwords

Only one SVC 19 is generated for each OPEN macro (MF=I or E)

A header precedes the list as follows:

DC C'zOPC'

DC AL2(no. of entries) The senior bit is off for MODE=24 and on for MODE=31

The list format and input to OPEN depend on MODE=

R1 points to the header

MODE=24 AL1(option),AL3(DCB/ACB address)

MODE=31 AL1(option),XL3'00',AL4(DCB/ACB address)

option=X'40' INPUT

option=X'20' OUTPUT

option=X'60' UPDATE

The last entry has the X'80' bit on in option

The option is ignored when opening an ACB

OPEN execution logic

OPEN execution logic is implemented as a Java routine

This logic consists of the following elements:

Action	Details
Determine type of parameter list	no.of entries senior bit off, MODE=24 no.of entries senior bit on, MODE=31
Determine if zVSAM V1 or V2	1st 4 bytes <> C'zOPC'=> ZVSAM V1 1st 4 bytes = C'zOPC'=> ZVSAM V2
loop over all entries in the parameter list	End-of-list is indicated in the option byte of the entry
- check address: ACB or DCB	First byte = X'A0' => ACB V1 First four bytes = C'zACB' => ACB V2 First four bytes = C'DCBV' => DCB Otherwise => Error
- if DCB invoke DCB open routine	OPEN logic for DCB is beyond the scope of this document
- if ACB validate ACB	ACBID <> X'A0' => Error ACBSTYP <> X'10' => Error ACBVER <> X'02' => Error ACB V1/V2 <> ZVSAM(n) parm => Error
- if ACB valid invoke VSAM open routine	
- next entry or end-of-loop	If bit 0 of an entry is on, terminate loop

OPEN logic for ACB handles a single ACB and proceeds as follows:

Action	Details
Check ACB status	If ACB already open, issue error and fail open Set ACBIOSFG
Copy ACB to newly created FCB	FCB is the java-equivalent of the ACB
Extract DDNAME	Copy ACBDDNM field from ACB/FCB
Find actual file name	Retrieve host variable with name matching ACBDDNM If not available: issue error and fail open
Validate against catalog	Find the file name in the catalog. If missing: issue error See note here
Issue OS open against file	Read-only if ACB specifies MACRF=IN Update/extend otherwise If unsuccessful issue error and fail open
Allocate buffer for prefix block	Save buffer address in FCB
Read first 4096 bytes into buffer	If read fails, issue error
Validate block header and footer	If BHDREYE <> C'HDR' issue error If BFTREYE <> C'FTR' issue error If BHDRSEQ# <> BFTRSEQ# issue error If BHDRVER <> X'02' issue error If BHDRSELF <> foxes issue error If BHDRPREV <> foxes issue error If BHDRNEXT <> foxes issue error If BHDRFLGS <> X'80' issue error

Action	Details
Validate prefix area	if PFXYEYE <> C'zPFX' issue error if filename <> PFXDNAM issue error if file's path <> PFXDPAT issue error if PFX_INDXX is on issue error
Validate counters area	if CTREYE <> C'zCTR' issue error
Validate prefix against catalog	Only if no errors detected thus far: compare cluster type compare lrecl compare blocksize compare key offset compare key length
Fail open on error	If any error was detected: - request OS to close the file - free the prefix buffer - set buffer address in FCB to zeros - fail the open request
Issue OS open against index file	If PFXXNAM@ is non-zero then open the indicated index file; read-only if ACB MACRF=IN for input/update/extend otherwise Read index header block and repeat all validations with the following modifications: - if PFX_INDXX is off rather than on issue an error
Fail open on error	If any error was detected: - request OS to close the files - free the prefix buffers - set buffer address in FCB to zeroes - fail the open request
Create data buffers	Based on ACBBUFND
Create index buffers	At least ACBBUFNI in total Exactly one for the root block At least 4 for each other index level
Open component	What is opened depends on what type of component the ACB points to A path may imply opening of the base cluster and/or AIXs Repeat the open process for each component File names and other info to be gathered from the catalog The table on the next page has the permutations of component types
Update ACB on OPEN completion	Set ACBIOFSG off Set ACBOPEN on Set addresses for ACBPFX, ACBXPFX, ACBBUFD, ACBBUFI Set ACBDTYPE If MACRF=AIX is specified, then ACB_AIX and ACB_BASE are set on

Note: The environment variables take the following form

SET ddname=drive:\path\catalog.filename

SET catalog=drive:\path

The ddname variable may only contain one dot

Implied OPEN table

This table has the permutations of component types, indented entries are implied processing

Open component	MACRF=IN	MACRF=OUT
Base	Opened for input	Opened for in/out
AIXs (UPGRADE=NO)	Not opened	Not opened
AIXs (UPGRADE=YES)	Not opened	Opened for in/out See Note 3 here
PATH (NOUPDATE) to Base	Opened for input No error if already open	Opened for in/out No error if already open
AIXs (UPGRADE=NO)	Not opened	Not opened
AIXs (UPGRADE=YES)	Not opened	Not opened See Note 1 here
PATH (UPDATE) to Base	Opened for input No error if already open	Opened for in/out No error if already open
AIXs (UPGRADE=NO)	Not opened	Not opened
AIXs (UPGRADE=YES)	Not opened	Opened for in/out See Note 3 here
PATH (NOUPDATE) to AIX See Note 4 here	Implied open of Base No error if already open See Note 2 here	Implied open of Base No error if already open See Note 2 here
AIXs (UPGRADE=NO)	AIX opened for input	AIX opened for input
AIXs (UPGRADE=YES)	Not opened See Note 1 here	Not opened See Note 1 here
PATH (UPDATE) to AIX See Note 4 here	Implied open of Base No error if already open See Note 2 here	Implied open of Base No error if already open See Note 2 here
AIXs (UPGRADE=NO)	AIX opened for input	AIX opened for input
AIXs (UPGRADE=YES)	Opened for in/out See Note 3 here	Opened for in/out See Note 3 here

Notes:

1. A NOUPDATE PATH means that the structures for AIXs on the upgrade set are not created
2. The Base is opened by zVSAM for input but has no associated ACB as it hasn't been opened by the app
3. All AIXs on the upgrade set are opened for in/out by zVSAM and may be updated
4. A PATH to an AIX ignores MACRF=IN/OUT

EXLST macro

The EXLST macro will generate an Exit List control block and initialize it according to the parameters specified on the macro invocation

The structure and layout of the generated EXLST are not part of the interface and are therefore not shown in this chapter. Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST=, TESTCB EXLST= and/or MODCB EXLST= to inspect, test, and/or modify the EXLST's content

All keywords on the EXLST macro are optional. Before the cluster is opened, all EXLST values can be modified using MODCB EXLST=, or by changing the EXLST directly. The latter is not recommended, as it is not guaranteed to be portable or compatible with future versions of zVSAM

The table below shows how the EXLST macro can be coded:

Opcode	Operand	Remarks
[label] EXLST	[AM=VSAM]	Designates this EXLST as a zVSAM EXLST
	[EODAD=(address[,mod])]	End-of-data exit routine
	[LERAD=(address[,mod])]	Logical error analysis routine
	[SYNAD=(address[,mod])]	Physical error analysis routine
	[JRNAD=(address[,mod])]	Not supported. Keyword is flagged with a warning message
	[UPAD=(address[,mod])]	Not supported. Keyword is flagged with a warning message
	[RLSWAIT=(address[,mod])]	Not supported. Keyword is flagged with a warning message

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

For GENCB MF=I, L or G, a missing address will generate zero and no error, IBM displays an error
It is assumed that the address will be made valid by a MODCB EXLST=
A missing mod will generate A

For GENCB MF=E, a missing address or mod means don't modify that parameter in the CBMR

Note: Although a null address maybe set in the EXLST, you cannot change an address to null with MODCB

EXLST macro parameters

EODAD= Optional parameter to specify the entry address of an exit that handles an end-of-data condition during sequential access
The routine address may be followed by a modifier. For details, please see below
The AMODE for the routine is encoded in the address using the common convention

LERAD= Optional parameter to specify the entry address of an exit that handles logic errors.
The routine address may be followed by a modifier. For details, please see below
The AMODE for the routine is encoded in the address using the common convention

SYNAD= Optional parameter to specify the entry address of an exit that handles physical errors.
The routine address may be followed by a modifier. For details, please see below
The AMODE for the routine is encoded in the address using the common convention

mod modifier, can optionally be specified after each routine address
Values: A or N for Active or Not-active. These are mutually exclusive
As long as the routine is not active it will not be called by zVSAM
The secondary modifier of L (for Load from Linklib) is not supported

Exit logic

This logic is only entered if any of the following conditions are raised:

End-of-data (EODAD)
Logical error (LERAD)
Physical error (SYNAD)

Action	Details
ACBEXLST has an address	No action if zero
Check that the exit is active	No action if inactive
Check that the address is not zero	No action if zero
Branch to the exit address	

CLOSE macro

A cluster needs to be closed after it has been processed. The close macro is used to close one or more clusters and/or one or more sequential files in a single call

Opcode	Operand	Remarks
[label] CLOSE	(entry[,entry]...)	Each cluster or file requires an entry of two parameters
	[MODE= <u>24</u> /31]	Residency mode of all control blocks involved Specify 31 if any reside above the line
	[TYPE=T]	Not supported – future option Keyword is flagged as ignored with a warning message
Entry format:	address,,	Address of ACB or DCB, followed by two commas to show that options are omitted
	[MF=I or omitted]	Use standard form of CLOSE
	[MF=L]	Use list form of CLOSE
	[MF=(E,address)]	Use execute form of CLOSE

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

CLOSE macro parameters

For ease of access a short summary follows here:

- entry The CLOSE macro accepts a list of entries. Each entry consists of two consecutive parameters: an address and an optional list of options
- address The address can be specified as an A-type address or as a register. If a register is coded the register number or name must be enclosed in parentheses. The address can be either the address of a DCB or the address of an ACB
- options Code as an omitted parameter
- MF=I or omitted If the MF parameter is omitted a close parmlist is generated inline, plus a call to the CLOSE SVC using the parmlist.
- MF=L With MF=L a close parmlist is generated inline
- MF=(E,address) Code to modify/populate the close parameter list at the indicated address, which may be a relocatable constant or a (register), plus a call to the CLOSE SVC using the parmlist

Note: IBMs CLOSE allows an MF=L to be built which can then be overwritten with an MF=E containing more entries than were originally created, thus causing storage violation.
zVSAM V2 will return R15=8 if this is attempted

IBMs CLOSE allows an MF=E to modify an MF=L with a different mode causing damage to the list
zVSAM V2 will return R15=8 if this is attempted

CLOSE logic

The close macro generates a close parameter list and/or an SVC 20 instruction to invoke the close routine. The syntax of the close macro is given [here](#)

The macro generates the following code:

MF variant	Generated Code
MF=L	Close parameter list data only
MF=(E,address)	1) Code to modify/populate the close parameter list at the indicated address, which may be a relocatable constant or a (register). 2) Code to invoke the close routine
MF=I or omitted	1) Close parameter list data (inline) 2) Code to invoke the close routine

Close parameter list entries have two different formats depending on the MODE parameter.

When MODE=24 then each entry is one fullword

When MODE=31 then each entry is two fullwords

Only one SVC 20 is generated for each CLOSE macro (MF=I or E)

A header precedes the list as follows:

DC C'zOPC'

DC AL2(no. of entries) The senior bit is off for MODE=24 and on for MODE=31

The list format and input to CLOSE depend on MODE=

R1 points to the header

MODE=24 AL1(option),AL3(DCB/ACB address)

MODE=31 AL1(option),XL3'00',AL4(DCB/ACB address)

option=0 except for the last entry when option=X'80'

CLOSE execution logic

Close involves lock and buffer management and may involve the closure of associated AIXs

CLOSE execution logic is implemented as a Java routine

This logic consists of the following elements:

Action	Details
Determine type of parameter list	no.of entries senior bit off, MODE=24 no.of entries senior bit on, MODE=31
Determine if zVSAM V1 or V2	1st 4 bytes <> C'zOPC'=> ZVSAM V1 1st 4 bytes = C'zOPC'=> ZVSAM V2
loop over all entries in the parameter list	End-of-list is indicated in the option byte of the entry
- check : ACB or DCB	First byte = X'A0' => ACB V1 First four bytes = C'zACB' => ACB V2 First four bytes = C'DCBV' => DCB Otherwise => Error
- if DCB invoke DCB close routine	CLOSE logic for DCB is beyond the scope of this document
- if ACB valid invoke VSAM close routine	
- next entry or end-of-loop	If bit 0 of an entry is on, terminate loop

CLOSE logic for ACB handles a single ACB and proceeds as follows:

Action	Details
Check ACB status	If ACB already closed, issue error and fail close
Check lock status	If any blocks in this dataset or any associated AIX are locked then wait until the locks are freed ???may need a timeout mechanism
Check buffer status	Free any read buffers Write any buffers marked as 'pending write' and then free them
Issue OS close against file	If unsuccessful issue error and fail close

RPL-based interfaces

The RPL is the primary interface for operations at the record level

A program can use multiple RPLs

An RPL must always point to an open ACB in order to specify a valid operation

RPL macro

The RPL macro will generate an RPL and initialize it according to the parameters specified on the macro invocation

Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL=, TESTCB RPL= and/or MODCB RPL= to inspect, test, and/or modify the RPL's content

All keywords on the RPL macro are optional. Before a request is issued, all RPL values can be modified using MODCB RPL=, or by changing the RPL directly. The latter is not recommended, as it is not guaranteed to be portable or compatible with future versions of zVSAM

The table below shows how the RPL macro can be coded

Opcode	Operand	Remarks
[label] RPL	[AM=VSAM]	Designates this RPL as a zVSAM RPL
	[ACB=address]	Address of an open ACB
	[AREA=address]	Address of a record area In Move mode the record is read into the area In Locate mode a to the record is moved into the area
	[AREALEN=value]	Length of record area or record
	[ARG=address]	Address of the search argument This is a key, a relative record number, or an RBA.
	[ECB=address]	Address of an ECB. Used with asynchronous requests See note here
	[KEYLEN=value]	Length of key value in ARG when a generic key search is requested
	[MSGAREA=address]	Address of message area
	[MSGLEN=value]	Length of message area
	[NXTRPL=address]	Address of the next RPL in the chain. RPLs can be chained together to request a series of operations in a single call to zVSAM
	[OPTCD=(keyword list)]	List of keywords specifying processing options See table below for valid keywords
	[RECLen=value]	Record length. Required when updating or adding records
	[TRANSID=value]	Not supported – future option. Keyword is flagged as ignored with a warning message

ECB note

RPLECB is an internal ECB if ECB= is not specified, or an external one if it is

The bit RPLOPT2_ECB is set for an external ECB

Supported options for the OPTCD parameter are listed below:

Keyword subset	Keyword	Remarks
[<u>ADR</u> <u>KEY</u>]	ADR	Addressed access to ESDS or KSDS (under review)
	KEY	Keyed access to KSDS or RRDS
	CNV	Not supported – future option. Keyword is flagged as ignored with a warning message
[<u>DIR</u> <u>SEQ</u> <u>SKP</u>]	DIR	Direct access to ESDS, KSDS, RRDS
	SEQ	Sequential access to ESDS, KSDS or RRDS
	SKP	Skip sequential access to KSDS or RRDS
[<u>ARD</u> LRD]	ARD	Access user-defined record location
	LRD	Access last record in the cluster
[<u>FWD</u> BWD]	FWD	Forward processing
	BWD	Backward processing
[<u>SYN</u> ASY]	SYN	Synchronous request
	ASY	Asynchronous request
[<u>NUP</u> <u>UPD</u> NSP]	NUP	Not for update
	UPD	For update
	NSP	Retain positioning for next sequential access
[<u>KEQ</u> KGE]	KEQ	Locate record with exact key match
	KGE	Locate record with exact key match, or next higher value
[<u>FKS</u> GEN]	FKS	Full key search
	GEN	Generic key search. KEYLEN required
[<u>MVE</u> LOC]	MVE	Move mode
	LOC	Locate mode
[<u>RBA</u> XRBA]	RBA	4-byte RBA values
	XRBA	8-byte extended RBA values
[NWAITX/WAITX]		Not supported – future option. Keyword is flagged as ignored with a warning message
[CR/NRI]		Not supported – future option. Keyword is flagged as ignored with a warning message

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

POINT macro
GET macro
PUT macro
ERASE macro
CHECK macro
ENDREQ macro
VERIFY macro

GENCB, MODCB, TESTCB and SHOWCB use of the CBMR

A CBMR is generated for all forms of these macros

Direct access to subfields in the CBMR is discouraged. Use SHOWCB, TESTCB and/or MODCB to inspect, test, and/or modify the content of an ACB, EXLST, or RPL. Use the appropriate MF= parameter on any of these macros to modify and/or use a CBMR

The CBMR consists of three parts: a header, a body, and a tail. The header has a fixed layout. The body consists of request-dependent fields and a list of verb codes. The tail contains all the data fields that go with the verb codes. Data fields can be 0, 4 or 8 bytes in length.

Verb codes X'01'-X'DF' have a 4-byte data field

Verb codes X'E0'-X'FF' have an 8-byte data field

All data fields in the tail are allocated consecutively, in the same order as the verbs that define their meaning

CBMR – header

The CBMR header identifies the type (ACB, EXLST, RPL, GENCB, MODCB, SHOWCB or TESTCB)

It also has details of any work area needed and a count of verbs in CBMRVRBS

CBMR – body

It's length is determined by the CBMRVRBS fields in the CBMR header

It contains one verb code for each specified parameter

CBMR – tail

The body is directly followed by the tail

It contains a data field of 4 or 8 bytes for each verb coded in the body, in the same sequence

The starting point of the tail can be found by adding the CBMRVRBS value to the end of the CBMR header

Its length can be calculated from the CBMRSIZE field, by subtracting both the header length and the CBMRVRBS field

GENCB, MODCB, TESTCB and SHOWCB use of MF=

All forms except MF=L generate executable code

MF=I or omitted Generates CBMR and invokes ZVSAM19C to process it

MF=L Generates CBMR inline

MF=(L,address) Generates CBMR inline and then moves it to address

MF=(L,address,label) as above and generates label equ size

MF=(E,address) Modifies the CBMR at address

Invokes ZVSAM19C to process the CBMR

MF=(G,address) Generates CBMR inline and then moves it to address

Invokes ZVSAM19C to process the CBMR

MF=(G,address,label) as above and generates label equ size

address can be label or reg, reg cannot be 0, 1, 14 or 15

reg is not permitted for MF=L

GENCB, MODCB, TESTCB and SHOWCB parameter types

For *abs expression* (called value in the macro definitions)

Parameter type	For MF=I/G/L	For MF=E
n	Permitted	Permitted
EQUated numeric value	Permitted, but not for LENGTH=	Permitted, but not for LENGTH=

For *address*

Parameter type	For MF=I/G/L	For MF=E
n	Permitted. See note 1 here	Permitted, but not for ERET= See note 1 here When n=0 see note 2 here
EQUated numeric value	Permitted, but not for LENGTH= See note 1 here	Permitted, but not for LENGTH= See note 1 here
ADCON-type address	Permitted	Permitted
Register form (reg)	Permitted, but not regs 0,1,14,15	Permitted, but not regs 0,1,14,15
Indirect form with ADCON (*,address)	Permitted for certain 8-byte fields See Note 3 here	Permitted for certain 8-byte fields See Note 3 here
Indirect form with disp(reg) (*,n(reg))	Permitted for certain 8-byte fields reg cannot be 0,1,14,15 See Note 3 here	Permitted for certain 8-byte fields reg cannot be 0,1,14,15 See Note 3 here

Note 1: The use of numeric values instead of an address may lead to accessing low storage and should be avoided

Note 2: An exception is TESTCB EODAD, JRNAD, LERAD and SYNAD where zero instead of an address means 'don't test the address'

Note 3: The following fields only support the indirect form in TESTCB

SDTASZ, STMST and all X* fields

The lack of proper syntax checking in the IBM macro can cause access to low storage or environmental destruction, so the following syntaxes are not allowed: (*,*) and (*,n)

GENCB BLK=ACB macro

This GENCB macro will generate ACBs and initialize or change them according to the parameters specified on the macro invocation. It is for this reason that all supported parameters and keywords of the ACB macro (as described above) are supported on the GENCB macro

Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB=, TESTCB ACB= and/or MODCB ACB= to inspect, test, and/or modify the ACB's content

Direct access to subfields in the CBMR is strongly discouraged

The GENCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] GENCB	BLK=ACB	Instructs GENCB to generate 1 or more ACBs
	[AM=VSAM]	Optional, no other values allowed
	[COPIES=1]	The number of identical ACBs to generate Specify a number between 1 and 65535
	[WAREA=address]	The work area where the ACBs are to be constructed
	[LENGTH=value]	Length of the work area in bytes If WAREA/LENGTH are omitted then storage is dynamically acquired and LOC=BELOW is the default
	[LOC=BELOW ANY]	Where GENCB is to allocate dynamically acquired storage if needed
	[other]	Any parameter supported on the ACB macro
	[MF=]	See the description of MF= here

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

WAREA= When WAREA is specified, LENGTH must be specified too
When WAREA is not specified, the CBMR handler allocates an area of storage
The address of this area whether via GETMAIN or WAREA is returned in R1
The length of the generated ACB(s) is returned in R0

LENGTH= Length in bytes of the area indicated by WAREA
When LENGTH is specified, WAREA must be specified as well

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful
R15=4 Reason Code=4 Invalid control block
R15=4 Reason Code=9 WAREA is too small
R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

GENCB BLK=EXLST macro

The GENCB macro with BLK=EXLST will generate or manipulate Exit Lists for use with ACBs and initialize or change them according to the parameters specified on the macro invocation. It is for this reason that all supported parameters and keywords of the EXLST macro (as described above) are supported on the GENCB macro when BLK=EXLST is specified

Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST=, TESTCB EXLST= and/or MODCB EXLST= to inspect, test, and/or modify the EXLST's content

Direct access to subfields in the CBMR is strongly discouraged

The GENCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks
[label] GENCB	BLK=EXLST	Instructs GENCB to generate one or more EXLSTs
	[AM=VSAM]	Optional, no other values allowed
	[COPIES= <u>1</u>]	The number of identical EXLSTs to generate Specify a number between 1 and 65535
	[WAREA=address]	The work area where the EXLSTs are to be constructed
	[LENGTH=value]	Length of the work area in bytes If WAREA/LENGTH are omitted then storage is dynamically acquired and LOC=BELOW is the default
	[LOC=BELOW ANY]	Where GENCB is to allocate dynamically acquired storage if needed
	[other]	Any parameter supported on the EXLST macro
	[MF=]	See the description of MF= here

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

WAREA= When WAREA is specified, LENGTH must be specified too
When WAREA is not specified, the CBMR handler allocates an area of storage
The address of this area whether via GETMAIN or WAREA is returned in R1
The length of the generated EXLST(s) is returned in R0

LENGTH= Length in bytes of the area indicated by WAREA.
When LENGTH is specified, WAREA must be specified as well

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful
R15=4 Reason Code=4 Invalid control block
R15=4 Reason Code=9 WAREA is too small
R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

GENCB BLK=RPL macro

The GENCB BLK=RPL macro generates or manipulates RPLs and initializes or changes them according to the parameters specified on the macro invocation. It is for this reason that all supported parameters and keywords of the RPL macro (as described above) are supported on the GENCB macro

Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL=, TESTCB RPL= and/or MODCB RPL= to inspect, test, and/or modify the RPL's content

Direct access to subfields in the CBMR is strongly discouraged

The GENCB RPL macro can be coded as follows:

Opcode	Operand	Remarks
[label] GENCB	BLK=RPL	Instructs GENCB to generate 1 or more RPLs
	[AM=VSAM]	Optional, no other values allowed
	[COPIES=1]	The number of identical RPLs to generate Specify a number between 1 and 65535
	[WAREA=address]	The work area where the RPLs are to be constructed
	[LENGTH=value]	Length of the work area in bytes If WAREA/LENGTH are omitted then storage is dynamically acquired and LOC=BELOW is the default
	[LOC=BELOW ANY]	Where GENCB is to allocate dynamically acquired storage if needed
	[other]	Any parameter supported on the RPL macro
	[MF=]	See the description of MF= here

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

WAREA= When WAREA is specified, LENGTH must be specified too
When WAREA is not specified, the CBMR handler allocates an area of storage
The address of this area whether via GETMAIN or WAREA is returned in R1
The length of the generated RPL(s) is returned in R0

LENGTH= Length in bytes of the area indicated by WAREA.
When LENGTH is specified, WAREA must be specified as well

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful
R15=4 Reason Code=4 Invalid control block
R15=4 Reason Code=9 WAREA is too small
R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

MODCB ACB= macro

The MODCB macro with ACB=address will modify an ACB according to the parameters specified on the macro invocation. It is for this reason that all parameters and keywords of the ACB macro (as described above) are supported on the MODCB macro when ACB=address is specified

Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB=, TESTCB ACB= and/or MODCB ACB= to inspect, test, and/or modify the ACB's content. See note [here](#)

Direct access to subfields in the CBMR is strongly discouraged

The MODCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] MODCB	ACB=address	Points MODCB to the ACB to be modified
	[AM= <u>VSAM</u>]	Optional, no other values allowed
	[other]	Any parameter supported on the ACB macro
	[MF=]	See the description of MF= here

All supported parameters are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual.

MACRF=

To clarify how MACRF works...

All supported subparameters have their own bit in CBMRACB_MACRF (currently 16),

Conflicts are MNOTEd, eg. bits for NIS and SIS cannot both be on

If MF=E is specified then the whole of CBMRACB_MACRF is replaced

When the ACB is modified...

For mutually exclusive parameters, the bit is turned on or off

For each non-exclusive parameter the appropriate bit is turned on, therefore it isn't possible to turn a non-exclusive bit off using MODCB, this has to be done manually

eg. When an ACB has MACRF=(OUT) which allows read and write functions it is not possible to change the ACB to read-only using MODCB

If this is needed code the instruction NI ACBMACR1,255-ACBOUT

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=4 Reason Code=12 MODCB was attempted on an open ACB

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

MODCB EXLST= macro

The MODCB macro with EXLST=address will modify an EXLST according to the parameters specified on the macro invocation. It is for this reason that all parameters and keywords of the EXLST macro (as described above) are supported on the MODCB macro when EXLST=address is specified

Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST=, TESTCB EXLST= and/or MODCB EXLST= to inspect, test, and/or modify the EXLST's content

Direct access to subfields in the CBMR is strongly discouraged

The MODCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks
[label] MODCB	EXLST=address	Points MODCB to the EXLST to be modified
	[AM= <u>VSAM</u>]	Optional, no other values allowed
	[other]	Any parameter supported on the EXLST macro
	[MF=]	See the description of MF= here

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=4 Reason Code=4 EXLST= does not point to an EXLST

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

MODCB RPL= macro

The MODCB macro with RPL=address will modify an RPL according to the parameters specified on the macro invocation. It is for this reason that all parameters and keywords of the RPL macro (as described above) are supported on the MODCB macro when RPL=address is specified

Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL=, TESTCB RPL= and/or MODCB RPL= to inspect, test, and/or modify the RPL's content

Direct access to subfields in the CBMR is strongly discouraged

The MODCB RPL macro can be coded as follows:

Opcode	Operand	Remarks
[label] MODCB	RPL=address	Points MODCB to the RPL to be modified
	[AM= <u>VSAM</u>]	Optional, no other values allowed
	[other]	Any parameter supported on the RPL macro
	[MF=]	See the description of MF= here

All supported parameters are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

ECB=

ECB= can be modified to zero or an address

If it's zero then RPLOPT2_ECB is reset (internal ECB)

If it's non-zero then RPLOPT2_ECB is set (external ECB)

OPTCD=

To clarify how OPTCD works...

All supported subparameters have their own bit in CBMRRPL_OPTCD (currently 22),

Conflicts are MNOTEd, eg. bits for FWD and BWD cannot both be on

If MF=E is specified then the whole of CBMRRPL_OPTCD is replaced

When the RPL is modified, then for each subset, RPLOPTn bits are turned on or off as appropriate

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

SHOWCB with no specified block type macro

The SHOWCB macro without a block will return length fields according to the parameters specified on the macro invocation in the order they are specified. Duplicates are permitted

Opcode	Operand	Remarks
[label] SHOWCB	[AM=VSAM]	Optional, no other values allowed
	AREA=address	Address of return area
	LENGTH=value	Size of return area in bytes
	FIELDS=(keyword list)	List of keywords indicating which fields to return
4	[MF=]	See the description of MF= here

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
ACBLEN	4	Length of ACB in bytes
EXLLEN	4	Length of EXLST in bytes
RPLLEN	4	Length of RPL in bytes

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=4 Reason Code=9 Length too small

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

SHOWCB ACB= macro

The SHOWCB macro with ACB=address will return ACB-related fields according to the parameters specified on the macro invocation in the order they are specified. Duplicates are permitted

Direct access to subfields in the ACB is discouraged. Use SHOWCB ACB=, TESTCB ACB= and/or MODCB ACB= to inspect, test, and/or modify the ACB's content

Direct access to subfields in the CBMR is strongly discouraged

The SHOWCB ACB macro can be coded as follows:

Opcode	Operand	Remarks
[label] SHOWCB	ACB=address	Points SHOWCB to the ACB to be queried
	[AM= <u>VSAM</u>]	Optional, no other values allowed
	AREA=address	Address of return area
	LENGTH=value	Size of return area in bytes
	[OBJECT= <u>DATA/INDEX</u>]	For KSDS: select data or index component
	FIELDS=(keyword list)	List of keywords indicating which fields to return
	[MF=]	See the description of MF= here

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
ACBLEN	4	Length of ACB in bytes
AVSPAC	4	Available space in data/index (last 4 bytes). Derived from CTRAVSPAC
BFRFND	4	value of buffer hits for data/index including LSR (last 4 bytes) Derived from CTRNBFRFND
BSTRNO	4	Initial value of strings for a path. Derived from ACBBSTNO
BUFND	4	value of data buffers specified in ACB. Derived from ACBBUFND
BUFNI	4	value of index buffers specified in ACB. Derived from ACBBUFNI
BUFNO	4	Number of data/index buffers allocated (last 4 bytes) Derived from CTRNBUFNO
BUFNOL	4	Number of data/index buffers allocated for LSR processing (returns zero)
BUFRDS	4	Number of data/index buffer reads (last 4 bytes). Derived from CTRNBUFRDS
BUFSP	4	Buffer space in bytes specified in ACB. Derived from ACBBUFSP
BUFUSE	4	Number of data/index buffers actually in use (last 4 bytes) Derived from CTRNBUFUSE
CDTASIZE	8	Size of a compressed dataset (returns zero)
CINV	4	Block size for data/index. Derived from PFXBLKSZ
CIPCA	4	CI's in CA (returns zero)
DDNAME	8	DDNAME specified in ACB. Derived from ACBDDNM
ENDRBA	4	Highest used RBA (last 4 bytes). Derived from CTRENDRBA
ERROR	4	Return code from last open/close operation. Derived from ACBERFLG

Keyword	Length	Remarks
EXLLEN	4	Length of EXLST in bytes
EXLST	4	address of EXLST, zero if none. Derived from ACBEXLST
FS	4	returns zero
HALCRBA	4	Highest allocated data/index RBA (last 4 bytes). Derived from CTRHALCRBA
HLRBA	4	For OBJECT=INDEX only, highest index block RBA Derived from CTRHLRBA
KEYLEN	4	Length of key field. Derived from PFXKYLEN
LEVEL	8	Address (4 bytes) and length (4 bytes) of field containing zVSAM version Derived from ACBVER
LOKEY	8	Address (4 bytes) of lowest key in the cluster + length (4 bytes) of key Derived from CTRLOKEY@ and PFXKYLEN Some requests may result in an error, see the list here
LRECL	4	Maximum data/index record length. Derived from PFXRCLEN
MAREA	4	Message area (returns foxes)
MLEN	4	Message length (returns zero)
NCIS	4	value of Block splits in the data component (last 4 bytes) Zero for OBJECT=INDEX. Derived from CTRNCIS
NDELRL	4	value of deleted records from the data component (last 4 bytes) Zero for OBJECT=INDEX. Derived from CTRNDELRL
NEXCP	4	value of I/O requests for the data/index components (last 4 bytes) Derived from CTRNEXCP
NEXT	4	value of extents of the data/index components (returns 1)
NINSR	4	value of records inserted for the data component (last 4 bytes) Zero for OBJECT=INDEX. Derived from CTRNINSR
NIXL	4	value of index levels for index component. Zero for OBJECT=DATA Derived from highest non-foxes PFXBLVLn
NLOGR	4	value of records in the data/index (last 4 bytes). Derived from CTRNLOGR
NRETR	4	value of records retrieved from the data component (last 4 bytes). Zero for OBJECT=INDEX. Derived from CTRNRETR
NSSS	4	value of control area splits for the data/index (returns zero)
NUIW	4	value of non-user writes (last 4 bytes). Derived from CTRNNUIW
NUPDR	4	value of updated records in the data/index components (last 4 bytes). Derived from CTRNUPDR
PASSWD	4	address to password, consisting of length (1 byte, binary) followed by the actual password value. Derived from ACBPASSW
RELEASE	8	Address (4 bytes) and length (4 bytes) of field containing zVSAM version Derived from ACBVER. Same as LEVEL
RKP	4	Relative Key Position, offset of key within logical record Derived from PFXKYOFF
RMODE31	4	0=None, 1=Buff, 2=CB, 3=All. Derived from ACBOFLGS

Keyword	Length	Remarks
RPLLEN	4	Length of RPL in bytes
SDTASIZE	8	Data size. Derived from CTRSDTASZ
SHRPOOL	4	SHRPOOL number. Derived from ACBSHRP
STMST	8	STCK of last close. Derived from CTRSTMST
STRMAX	4	Max value of concurrently active strings (last 4 bytes). Derived from CTRSTRMAX
STRNO	4	Max value of allocated strings. Derived from ACBSTRNO
UIW	4	value of user writes for data/index (last 4 bytes). Derived from CTRNUIW
XAVSPAC	8	AVSPAC when value may exceed 4GB
XBFRFND	8	BFRFND when value may exceed 4GB
XBUFNO	8	BUFNO when value may exceed 4GB
XBUFRDS	8	BUFRDS when value may exceed 4GB
XBUFUSE	8	BUFUSE when value may exceed 4GB
XENDRBA	8	ENDRBA when value may exceed 4GB
XHALCRBA	8	HALCRBA when value may exceed 4GB
XHLRBA	8	HLRBA when value may exceed 4GB
XNCIS	8	NCIS when value may exceed 4GB
XNDEL	8	NDEL when value may exceed 4GB
XNEXCP	8	NEXCP when value may exceed 4GB
XNINSR	8	NINSR when value may exceed 4GB
XNLOGR	8	NLOGR when value may exceed 4GB
XNRETR	8	NRETR when value may exceed 4GB
XNUIW	8	NNUIW when value may exceed 4GB
XNUPDR	8	NUPDR when value may exceed 4GB
XSTRMAX	8	STRMAX when value may exceed 4GB
XUIW	8	UIW when value may exceed 4GB

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=1 ACBPFX or ACBXPFX are zero

(X)HLRBA requested and OBJECT=DATA

For fields that have 8-byte values (eg. XHLRBA) the 4-byte version is requested but the 1st four bytes are not zero

CTRLOKEY@ is foxes for:

non-KSDS

KSDS index

KSDS data but empty

R15=4 Reason Code=4 Invalid control block

R15=4 Reason Code=9 Length too small

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

SHOWCB EXLST= macro

The SHOWCB macro with EXLST=address will return EXLST-related fields according to the parameters specified on the macro invocation in the order they are specified. Duplicates are permitted

Direct access to subfields in the EXLST is discouraged. Use SHOWCB EXLST=, TESTCB= EXLST and/or MODCB EXLST= to inspect, test, and/or modify the EXLST's content

Direct access to subfields in the CBMR is strongly discouraged

The SHOWCB EXLST= macro can be coded as follows:

Opcode	Operand	Remarks
[label] SHOWCB	EXLST=address	Points SHOWCB to the EXLST to be queried
	[AM= <u>VSAM</u>]	Optional, no other values allowed
	AREA=address	Address of return area
	LENGTH=value	Size of return area in bytes
	FIELDS=(keyword list)	List of keywords indicating which fields to return
	[MF=]	See the description of MF= here

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
ACBLEN	4	Length of ACB in bytes
EODAD	4	End-of-data exit routine address
EXLLEN	4	Length of EXLST in bytes
JRNAD	4	Supported here, but as it's not supported by other macros, zero is returned
LERAD	4	Logical error analysis routine address
RPLLEN	4	Length of RPL in bytes
SYNAD	4	Physical error analysis routine address

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation. For details, please refer to the relevant IBM manual

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=4 Reason Code=9 Length too small

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

SHOWCB RPL= macro

The SHOWCB macro with RPL=address will return RPL-related fields according to the parameters specified on the macro invocation in the order they are specified. Duplicates are permitted

Direct access to subfields in the RPL is discouraged. Use SHOWCB RPL=, TESTCB RPL= and/or MODCB RPL= to inspect, test, and/or modify the RPL's content

Direct access to subfields in the CBMR is strongly discouraged

The SHOWCB RPL= macro can be coded as follows:

Opcode	Operand	Remarks
[label] SHOWCB	RPL=address	Points SHOWCB to the RPL to be queried
	[AM= <u>VSAM</u>]	Optional, no other values allowed
	AREA=address	Address of return area
	LENGTH=value	Size of return area in bytes
	FIELDS=(keyword list)	List of keywords indicating which fields to return
	[MF=]	See the description of MF= here

Supported options for the FIELDS parameter are listed below:

Keyword	Length	Remarks
ACB	4	Address of ACB
ACBLEN	4	Length of ACB in bytes
AIXPC	4	Alternate index count. Derived from PFXAIXN
AREA	4	Address of record buffer
AREALEN	4	Size of record buffer in bytes
ARG	4	Address of search argument field
ECB	4	Address of ECB
EXLLEN	4	Length of EXLST in bytes
FDBK	4	Feedback code for the last request
FTNCD	4	Function code
KEYLEN	4	Length of key, for use with OPTCD=GEN
MSGAREA	4	Address of message area (returns foxes)
MSGLEN	4	Length of message area (returns zero)
NXTRPL	4	Address of next RPL
RBA	4	4-byte RBA of last record processed (ESDS only, otherwise zero)
RECLLEN	4	Length of current record
RPLLEN	4	Length of RPL in bytes
TRANSID	4	Transaction id (returns foxes)
XRBA	8	8-byte RBA of last record processed (ESDS only, otherwise zero)

All supported parameters and keywords are implemented compatibly with IBM's VSAM implementation
For details, please refer to the relevant IBM manual

Return (R15) and Reason (R0) Codes:

R15=0	Reason Code=n/a	Successful
R15=4	Reason Code=1	AIXPC or RPLDACB are zero
R15=4	Reason Code=4	Invalid control block
R15=4	Reason Code=9	Length too small
R15=8	Reason Code=n/a	An attempt was made to update a CBMR with a field not previously created

TESTCB Introduction

Only a single test can be specified on each TESTCB invocation

In all cases the value or address supplied in the macro is compared to a constant or a value in a control block

Many parameters have been added in zVSAM V2 to bring it in line with SHOWCB, these are in bold

An extra column is provided in the charts below to indicate the type of condition code that could be returned, the notation NE=LO (etc) is used to indicate that although LO may be returned the preferred test is for EQ/NE

Where a parm may have subparameters, then all must be true for an EQ to be returned, if unsupported parms or subparameters are specified then NE=LO is returned, there are more details against the parm itself

It is highly recommended that a branch table is placed after the TESTCB to capture any error conditions, the condition code is unpredictable if an error does occur

Example:

```
TESTCB ACB=MYACB,NCIS=20,MF=I
B      *+4(R15)
J      OK
J      ERR04
J      ERR08
OK     DS      0H
```

IBMs TESTCB is very badly syntax checked, zVSAM V2 has tightened the rules

It's very unlikely that imported code will result in unexpected errors, the author would like to know of any

More details against specific parms

TESTCB with no specified block type macro

The TESTCB without a block macro can be coded as follows:

Opcode	Operand	Remarks	Conditions returned
[label] TESTCB	[AM=VSAM]	Optional, no other values allowed	
	[ERET=address]	Address of error handling routine	
	ACBLEN=value	ACB length	EQ LO HI
	RPLLEN=value	RPL length	EQ LO HI
	EXLLEN=value	EXLST length	EQ LO HI
	[MF=]	See the description of MF= here	

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

TESTCB ACB= macro

The TESTCB ACB macro can be coded as follows:

Opcode	Operand	Remarks	Conditions returned
[label] TESTCB	ACB=address	Points TESTCB to the ACB to be tested	
	[AM=VSAM]	Optional, no other values allowed	
	[ERET=address]	Address of error handling routine	
	[OBJECT=DATA/INDEX]	Select data or index component	
	ACBLEN=value	ACB length	EQ LO HI
	RPLLEN=value	RPL length	EQ LO HI
	EXLLEN=value	EXLST length	EQ LO HI
	ATRB=(keyword list)	List of keywords indicating attributes to test All subparameters have to be true for EQ See Note 1 here	EQ NE=HI
	AVSPAC=value	Available space in data/index (last 4 bytes) From CTRAVSPAC	EQ LO HI
	BFRFND=value	Buffer hits for data/index including LSR (last 4 bytes). From CTRNBFRFND	EQ LO HI
	BSTRNO=value	Initial value of strings for a path From ACBBSTNO	EQ LO HI
	BUFND=value	value of data buffers. From ACBBUFND	EQ LO HI
	BUFNI=value	value of index buffers. From ACBBUFNI	EQ LO HI
	BUFNO=value	value of I/O Buffers (last 4 bytes) From CTRNBUFNO	EQ LO HI
	BUFRDS=value	Number of data/index buffer reads (last 4 bytes). From CTRNBUFRDS	EQ LO HI
	BUFSP=value	Buffer space in bytes. From ACBBUFSP	EQ LO HI
	BUFUSE=value	Number of data/index buffers actually in use (last 4 bytes). From CTRNBUFUSE	EQ LO HI
	CINV=value	Block size in bytes. From PFXBLKSZ	EQ LO HI
	DDNAME=string	DDNAME. From ACBDDNM	EQ LO HI
	ENDRBA=value	Highest used RBA (last 4 bytes). From CTRENRBA	EQ LO HI
	ERROR=value	Return code from last open/close operation From ACBERFLG	EQ LO HI
	EXLST=address	EXLST address. From ACBEXLST	EQ LO HI
	FS=value	Compares to zero	EQ LO HI
	HALCRBA=value	Highest allocated data/index RBA (last 4 bytes). From CTRHALCRBA	EQ LO HI
	HLRBA=value	For OBJECT=INDEX only, highest index block RBA. From CTRHLRBA	EQ LO HI

Opcode	Operand	Remarks	Conditions returned
	KEYLEN=value	Length of key field. From PFXKYLEN	EQ LO HI
	LRECL=value	Logical Record Length. From PFXRCLEN	EQ LO HI
	MACRF=(keyword list)	List of keywords for processing options All subparameters have to be true for EQ From ACBMACRn. See Note 2 here	EQ NE=HI
	NCIS=value	value of block splits (last 4 bytes) Compares to zero for OBJECT=INDEX From CTRNCIS	EQ LO HI
	NDELR=value	value of deleted records (last 4 bytes) Compares to zero for OBJECT=INDEX From CTRNDELR	EQ LO HI
	NEXCP=value	value of I/O requests (last 4 bytes) From CTRNEXCP	EQ LO HI
	NEXT=value	value of extents (last 4 bytes) From CTRNEXT (always 1)	EQ LO HI
	NINSR=value	value of inserted records (last 4 bytes) Compares to zero for OBJECT=INDEX From CTRNINSR	EQ LO HI
	NIXL=value	value of index levels Compares to zero for OBJECT=DATA From PFXBLVLn	EQ LO HI
	NLOGR=value	value of records (last 4 bytes) From CTRNLOGR	EQ LO HI
	NRETR=value	value of records retrieved (last 4 bytes) Compares to zero for OBJECT=INDEX From CTRNRETR	EQ LO HI
	NSSS=value	Compares to zero	EQ LO HI
	NUIW=value	value of non-user writes (last 4 bytes). From CTRNNUIW	EQ LO HI
	NUPDR=value	value of updated records (last 4 bytes) From CTRNUPDR	EQ LO HI
	OFLAGS=OPEN	Successful OPEN. From ACBOFLGS	EQ NE=HI
	OPENOBJ=PATH BASE AIX	ACB represents Path, Base or AIX From ACBDTYPE	EQ NE=HI
	PASSWD=address	address of 1-byte length, password From ACBPASSW	EQ LO HI
	RKP=value	offset of key field within record From PFXKYOFF	EQ LO HI
	SHRPOOL=value	SHRPOOL number. From ACBSHRP	EQ LO HI
	SDTASZ=address	Data size. From CTRSDTASZ	EQ LO HI

Opcode	Operand	Remarks	Conditions returned
	STMST=address	Address of system timestamp field From CTRSTMST	EQ LO HI
	STRMAX=value	Max. value of concurrently active strings (last 4 bytes). Derived from CTRSTRMAX	EQ LO HI
	STRNO=value	Max. value of parallel requests From ACBSTRNO	EQ LO HI
	UIW=value	value of user writes (last 4 bytes). From CTRNUIW	EQ LO HI
	XAVSPAC=value	AVSPAC when value may exceed 4GB	EQ LO HI
	XBFRFND=value	BFRFND when value may exceed 4GB	EQ LO HI
	XBUFNO=value	BUFNO when value may exceed 4GB	EQ LO HI
	XBUFRDS=value	BUFRDS when value may exceed 4GB	EQ LO HI
	XBUFUSE=value	BUFUSE when value may exceed 4GB	EQ LO HI
	XENDRBA=value	ENDRBA when value may exceed 4GB	EQ LO HI
	XHALCRBA=value	HALCRBA when value may exceed 4GB	EQ LO HI
	XHLRBA=value	HLRBA when value may exceed 4GB	EQ LO HI
	XNCIS=value	NCIS when value may exceed 4GB	EQ LO HI
	XNDELRL=value	NDELRL when value may exceed 4GB	EQ LO HI
	XNEXCP=value	NEXCP when value may exceed 4GB	EQ LO HI
	XNEXT=value	NEXT when value may exceed 4GB	EQ LO HI
	XNINSR=value	NINSR when value may exceed 4GB	EQ LO HI
	XNLOGR=value	NLOGR when value may exceed 4GB	EQ LO HI
	XNRETR=value	NRETR when value may exceed 4GB	EQ LO HI
	XNUIW=value	NNUIW when value may exceed 4GB	EQ LO HI
	XNUPDR=value	NUPDR when value may exceed 4GB	EQ LO HI
	XSTRMAX=value	STRMAX when value may exceed 4GB	EQ LO HI
	XUIW=value	UIW when value may exceed 4GB	EQ LO HI
	[MF=]	See the description of MF= here	

Note 1: The syntax rules for ATRB have been tightened in zVSAM V2

ESDS, KSDS, LDS, RRDS and VRRDS are considered mutually exclusive, PFXFFLGS is tested

For SPAN or UNQ, PFXRFLGS is tested

If COMPRESS, REPL, SSWD or WCK are included then NE=HI is returned

Note 2: If any of the following keywords are used then NE=HI will be returned

CNV, CFX, NFX, DDN, DSN, LEW, NLW, NRS, RST, RLS, NUB, UBF, NCI, ICI

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=1 This parameter requires the dataset to be open
(X)HLRBA requested and OBJECT=DATA
For fields that have 8-byte values (eg. XHLRBA) the 4-byte version is
requested but the 1st four bytes are not zero

R15=4 Reason Code=4 Invalid control block

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

TESTCB EXLST= macro

If mod L is specified then NE=LO is returned

The TESTCB EXLST macro can be coded as follows:

Opcode	Operand	Remarks	Conditions returned
[label] TESTCB	EXLST=address	Points TESTCB to the EXLST to be tested	
	[AM=VSAM]	Optional, no other values allowed	
	ERET=address	Address of error handling routine	
	ACBLEN=value	ACB length	EQ LO HI
	RPLLEN=value	RPL length	EQ LO HI
	EXLLEN=value	EXLST length	EQ LO HI
	EODAD=(address[,mod])	End-of-data exit address If address is zero or omitted and no mod..... If address is zero and mod (only mod is tested).. If address is not zero and no mod..... If address is not zero and mod.....	EQ EQ NE=LO EQ LO HI EQ NE=LO
	JRNAD=	Journal exit address (allowed but not supported)	NE=LO
	LERAD=(address[,mod])	Logical error analysis address If address is zero or omitted and no mod..... If address is zero and mod (only mod is tested).. If address is not zero and no mod..... If address is not zero and mod.....	EQ EQ NE=LO EQ LO HI EQ NE=LO
	SYNAD=(address[,mod])	Physical error analysis address If address is zero or omitted and no mod..... If address is zero and mod (only mod is tested).. If address is not zero and no mod..... If address is not zero and mod.....	EQ EQ NE=LO EQ LO HI EQ NE=LO
	[MF=]	See the description of MF= here	

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=4 Invalid control block

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

TESTCB RPL= macro

The TESTCB RPL macro can be coded as follows:

Opcode	Operand	Remarks	Conditions returned
[label] TESTCB	RPL=address	Points TESTCB to the RPL to be tested	
	[AM=VSAM]	Optional, no other values allowed	
	ERET=address	Address of error handling routine	
	ACBLEN=value	ACB length	EQ LO HI
	RPLLEN=value	RPL length	EQ LO HI
	EXLLEN=value	EXLST length	EQ LO HI
	ACB=address	ACB address	EQ LO HI
	AIXFLAG=AIXPKP	AIX pointer type. From RPLAIXID	EQ is RBA NE=HI is Key
	AIXPC=value	no. of AIXs in upgrade set. From PFXAIXN	EQ LO HI
	AREA=address	address of record area. From RPLAREA	EQ LO HI
	AREALEN=value	length of record area. From RPLAREAL	EQ LO HI
	ARG=address	Address of ARG. From RPLARG	EQ LO HI
	ECB=address	Address of ECB. From RPLECB	EQ LO HI
	FDBK=value	Feedback code of the last request From RPLERRCD	EQ LO HI
	FTNCD=value	Function code. From RPLCMPON	EQ LO HI
	IO=COMPLETE	I/O is complete. From RPLECB,RPLPOST	EQ is complete NE=LO is not
	KEYLEN=value	Length of key field	EQ LO HI
	MSGAREA=address	Address of message area. From RPLMSGAR	EQ LO HI
	MSGLEN=value	Length of message area. From RPLMSGLN	EQ LO HI
	NXTRPL=address	Address of next RPL. From RPLNXTRP	EQ LO HI
	OPTCD=(keyword list)	List of keywords indicating attributes to test All subparameters have to be true for EQ From RPLOPTn	EQ NE=HI
	RBA=value	Current RBA (last 4 bytes). From RPLCXRBA	EQ LO HI
	RECLEN=value	Record Length. From RPLRECLN	EQ LO HI
	TRANSID=value	Allowed but not supported	EQ
	XRBA=value	Current RBA. From RPLCXRBA	EQ LO HI
	[MF=]	See the description of MF= here	

Return (R15) and Reason (R0) Codes:

R15=0 Reason Code=n/a Successful

R15=4 Reason Code=1 This parameter requires the dataset to be open
For fields that have 8-byte values (eg. XRBA) the 4-byte version is
requested but the 1st four bytes are not zero

R15=4 Reason Code=4 Invalid control block

R15=8 Reason Code=n/a An attempt was made to update a CBMR with a field not previously created

Catalog management

This is where all meta-data about the zVSAM components are kept and where the relations between zVSAM components are defined. Catalogs are currently created as static assembled modules
Extended catalogs contained in datasets will be considered in a future release

The catalog will hold at least:

- file name
- pointer to index file
- pointers to all related AIX clusters
- LRECL
- record type (F, V, FS, VS)
- type of component (ESDS, KSDS, RRDS, AIX)
- freeblocks (during load, between blocks)
- freespace (during load, within blocks)
- Physical Block size (aka CI-size, 512 bytes to 16MB)

For a complete list of catalog components please see the [“z390_zVSAM_Catalog_User_Guide”](#)

Physical structure of the files

Basic Concepts

Files, Blocks, Records

The logical unit of access or storage is the record. Yet the unit for any given I/O operation is the block. Block sizes may vary from 512 bytes to 16MB. Each block holds up to 255 records. For any given cluster component, choosing an appropriate block size is important. Block size can greatly affect not only performance, but also both internal and external storage consumption

A cluster consists of one or more files that belong together and should be managed together. Whether you take a backup, perform a restore, or perform other administrative tasks, the files that make up a cluster should be managed alike. When creating a backup copy of a cluster or restoring a cluster, make sure no other processes try to access the data at the same time

zVSAM implements a number of checks and balances to prevent inadvertent access to data that may have been compromised. Names and locations of files are managed. Tampering with files or file attributes may render the cluster unusable

As a result, it is not possible to rename a zVSAM cluster or file. Unload and reload your cluster in order to move the data or to assign a different name to cluster or file

Just like files in a cluster belong together and should be managed together, clusters in a sphere are logically connected and should be managed together. Again, failing to manage the files in a correct and comprehensive manner may render your data inaccessible

Cluster types and Cluster Components

Each cluster consists of a data component and an index component as follows:

Data and Index Components per Cluster type	
Cluster type	Index content
ESDS	Index on XRBA
KSDS	Index on key value
RRDS	Index on RRN
AIX	Index on alternate key value

Record Formats

In zVSAM we support the following record formats:

Record Formats	
Format	Properties
F	Fixed. All records have the same length Records never span a Block boundary
FS	Fixed Spanned. All records have the same length Records expected to span a Block boundary
V	Variable. Records have varying lengths Records never span a Block boundary
VS	Variable Spanned. Records have varying lengths Records may or may not span a Block boundary

For ESDS, KSDS, and RRDS all record types are supported.

For AIX only F and VS record formats are supported: F for unique, and VS for non-unique indexes

Supported Record Formats				
Cluster Type	F	FS	V	VS
ESDS	Y	Y	Y*	Y*
KSDS	Y	Y	Y	Y
RRDS	Y	Y*	Y	Y*
AIX - unique	Y	N	N	N
AIX - non-unique	N	N	N	Y
* zVSAM extension				

For a unique AIX each record holds an alternate key value plus the primary key (KSDS) or XLRA (ESDS) of the associated record in the cluster's data component. This fixed configuration dictates a record type of F

For a non-unique AIX each record holds an alternate key value and as many primary keys (KSDS) or XLRA (ESDS) of associated records in the cluster's data component as there are records holding that specific alternate key value. The table of primary keys may vary in length from 1 to very large numbers. No block size is guaranteed to be large enough to hold the largest possible index record, therefore a record type of VS is mandated. When a non-unique index record needs to be split into segments, no primary key value or XLRA is ever split; i.e. only an exact number of these reside within a single segment of the record

Supported Index-types			
Cluster Type	Primary - Unique	AIX - unique	AIX - Non-unique
ESDS	Y*	Y	Y
KSDS	Y	Y	Y
RRDS	Y*	N	N
* zVSAM extension			

File Structure

Physical files

All zVSAM data is stored in physical files, as defined to the operating system

Each component consists of one file. This file is formatted as a zVSAM file, the structure of which is explained in the next set of chapters

Please note: the hosting operating system may impose a limit on physical file size and not every host OS supports a physical file spanning a volume boundary of the storage device(s). Therefore, to support clusters that exceed the maximum size of a single physical file, in the future we may need to support clusters that consist of multiple files

Structure of physical files

Every zVSAM file has a block size. The block being the basic unit of I/O. The first block of every file is the prefix block, which is always 4096 bytes in size. The prefix block holds information about the cluster, its data, and its structure

Data in the prefix block are not accessible to user programs. However, selected fields in the prefix block can be queried using a SHOWCB ACB= request

All Data and Index blocks in the file have a user-defined blocksize (DATABLOCKSIZE= and INDEXBLOCKSIZE=). The file is assumed to logically begin with the first block after the prefix block
There are 5 types of blocks that may occur in zVSAM files:

- 1) Prefix block – one for each file, being the first 4096 bytes of every file
- 2) Spacemap block – used to manage free space in the file
- 3) Data block – used to hold user data, or AIX data records (in an AIX only)
- 4) Index block – used to hold index information
- 5) ELIX block – used to index segmented (read: large) non-unique AIX records

Every block has an internal structure consisting of a block header, a list of records, a block body and a block footer. The block header and footer have a fixed structure. The list of records has a variable length. The block body contains record data and/or free space

Each of the 5 block types is explained in more detail below

Block Header Structure

Every block has a block header (ZVSAMHDR). All block headers have the same structure

BHDRSEQ# is incremented by one every time the block is written out to the file

The footer area contains a comparable field: BFTRSEQ#. Together they guard against incomplete writes

BHDRXLVL indicates the index level. Zero is the leaf level. Index blocks are chained by level. That is, for every index level in use there is a pair of s in the prefix block (PFXBLVLn/PFXELVLn) that starts and ends the chain for that level

BHDRSELF contains the block's own XLRA. This helps to guard against misdirected reads and/or writes. BHDRNEXT/BHDRPREV point to the next and previous block on the chain. Which chain this is, depends on the BHDRFLAG setting, and, if this is an index block, by the BHDRXLVL value

For the prefix block, these two fields are set to foxes

Segmented records are a special case. Segments of a segmented record never share their block with other data. The block holding the first segment is part of the data chain. A block holding a non-first segment is part of the segment chain. A block that holds a record's first segment has an SPX pointing to the block holding the next segment. Subsequent segments are retrieved by following the SPXs to the last segment of the record

The Segment chain starting at PFXBSEGM and ending at PFXESEGM has no role in processing a spanned dataset but just provides an extra integrity check

Block Footer Structure

Every block has a block footer zVSAMFTR). All block footers have the same structure

BFTRSEQ# is incremented by one every time the block is written out to the file

The header area contains a comparable field: BHDRSEQ#. Together they guard against incomplete writes

Prefix Block

The prefix block (ZVSAMPFX) consists of the first 4096 bytes of every physical file. It contains meta-data defining the file and its attributes. It also contains various counters

The prefix block consists of a block header immediately followed by the prefix area

The prefix block also contains other data fields, these are addressed from the prefix area

The prefix block ends with a block footer. A record list is not present on the prefix block

There are various fields in the prefix area. These point to fields allocated elsewhere in the prefix block. Their exact addresses on the prefix block may vary

The PFXDPAT@, PFXDNAM@, PFXXPAT@, PFXXNAM@ all point to a halfword-prefixed string
PFXDVOL@ and PFXXVOL@ contain foxes (future option)

The Counters area (ZVSAMCTR) directly follows the Prefix area, it is doubleword aligned
This area is expected to move into the catalog dataset in a future release

The overall structure of the prefix block would look something like this (areas not to scale):

Block Header	Prefix area - part 1	
Prefix area - part 2		
Counters area	Free space	
Free space		
Free space	Other fields addressed from Prefix area	Block Footer

Prefix Block chain summary

The following table summarizes the way that blocks in the file are chained from the prefix block

The prefix block doesn't reside on any chain

Block Type	Beginning of chain	End of chain
Prefix	foxes	foxes
Spacemap	PFXBMAP	PFXEMAP
Data (in use and free)	PFXBDATA	PFXEDATA
Data (non-first segments)	PFXBSEGM	PFXESEGM
Index (in use and free)	PFXBLVLn	PFXELVLn

Counters Area and its maintenance

All fields are 8 bytes except CTRAVGRL which is 4 bytes

Counter	Data/Index	Initialized by zREPRO	Maintenance
CTRAVGRL	Both	Yes For fixed, =PFXRECLN even if the dataset is empty For variable, calculated or zero if the dataset is empty	For variable files only: At CLOSE, calculate CTRTOTRL/CTRNLGR
CTRAVSPAC	Both	Yes	For every block update use the old and new BHDFREE to increase/decrease this value
CTRHLCRBA	Both	Yes	Updated when blocks are added to the end of the dataset component or when the existing HALCRBA block has all records deleted It's the block RBA+1 (XLRA+256) of the last data or level 0 index block containing records
CTRHLRBA	Index only	Yes	Block RBA of PFXROOT Update if it changes
CTRENDRBA	Both	Yes	Updated when blocks are added to the end of the dataset component It's the block RBA+1 (XLRA+256) of the last data or level 0 index block
CTRNBFREFND	Both	No	+1 for each LSR buffer read
CTRNBUFNO	Both	No	+1 for each buffer allocated
CTRBUFUSE	Both	No	+1 for each buffer used
CTRBUFRDS	Both	No	+1 for each buffer read
CTRNCIS	Both	No	+1 for each block split
CTRNDCLR	Both	No	KSDS or RRDS: +1 for each record delete
CTRNECP	Both	No	+1 for each physical block read/write
CTRNEXT	Both	Yes	Always 1, not maintained
CTRNINSR	Both	No	+1 for each record added For RRDS, any empty slots added to the end are not counted
CTRNLGR	Both	Yes	+1 for each record added -1 for each record deleted For RRDS, any empty slots added to the end are not counted For Index, all records in all levels are counted
CTRNRETR	Both	No	+1 for each record read

CTRNNUIW	Both	No	+1 for each maintenance write for block splits, chain repair, segment, spacemap and ELIX block management
CTRNUUPDR	Both	No	+1 for each record update
CTRSDTASZ	Both	Yes	+block size for each block added
CTRSTMST	Both	Yes	Write STCK value at CLOSE
CTRSTRMAX	Both	No	+1 for each string created
CTRNUIW	Both	No	+1 for each user-requested block write
CTRTOTRL	Data only	Yes	Maintained for variable files only +record size for each record added -record size for each record deleted SPX is not included RLF is included Adjusted for change to variable length For RRDS, empty slots are not included
CTRLOKEY	Data only	Yes	KSDS only Update when a lower key is added or this key is deleted

Spacemap Blocks

Spacemap blocks (ZVSAMMAP) are used to manage available free space in a component. Each spacemap block has a size that matches the blocksize of all other blocks (except possibly the prefix block) in the component

A component will hold as many spacemap blocks as needed to map all of its allocated blocks, including all spacemap blocks but excluding the prefix block. Whenever a single spacemap block is not enough, the spacemap blocks are chained together by means of the BHDRNEXT/BHDRPREV s in the block header area. The spacemap chain starts/ends from the prefix block, fields PFXBMAP/PFXEMAP

When a single spacemap block suffices, PFXBMAP and PFXEMAP will both point to that block. Each spacemap block consists of a block header immediately followed by the spacemap area, which in turn is followed directly by the block footer. No free space exists on a spacemap block. Thus, a spacemap block may indicate blocks that do not exist in the dataset. The bit settings for blocks beyond the PFXHXLRA should all be zero to indicate an unallocated block. zVSAM is aware that any block beyond PFXHXLRA needs to be created and initialized before it can be allocated

Conceptually, the overall structure of a spacemap block would look something like this (areas not to scale):

Block Header	Spacemap area
Spacemap area	
Spacemap area	
Spacemap area	Block Footer

Record List Structure (RPTR)

Every block that contains data records contains a record list (ZVSAMRPT). Records are accessible only through their Record or RPTR. Every entry in the list corresponds with a single record on the block. The last byte of the record's XLRA is the index into the Record List. Index value of X'00' is reserved for block s; values X'01' through X'FF' inclusive are usable as RPTR index values. The difference of 1 always needs to be taken into account when indexing the RPTR list

The RPTR list always follows the block header directly

The number of entries on the RPTR list varies with the number of records stored on the block (BHDR#REC) and is terminated with an entry of foxes to mark the end of the list.

When RPTR_END is set, RPTRREC@ is set to foxes

RPTR_ACT and RPTR_MTY are mutually exclusive. Either one must be set, otherwise the RPTR list is compromised and data access will fail

RPTR_MTY indicates an empty RRDS slot

Segment Prefix (SPX)

All segments begin with a segment prefix (ZVSAMSEG)

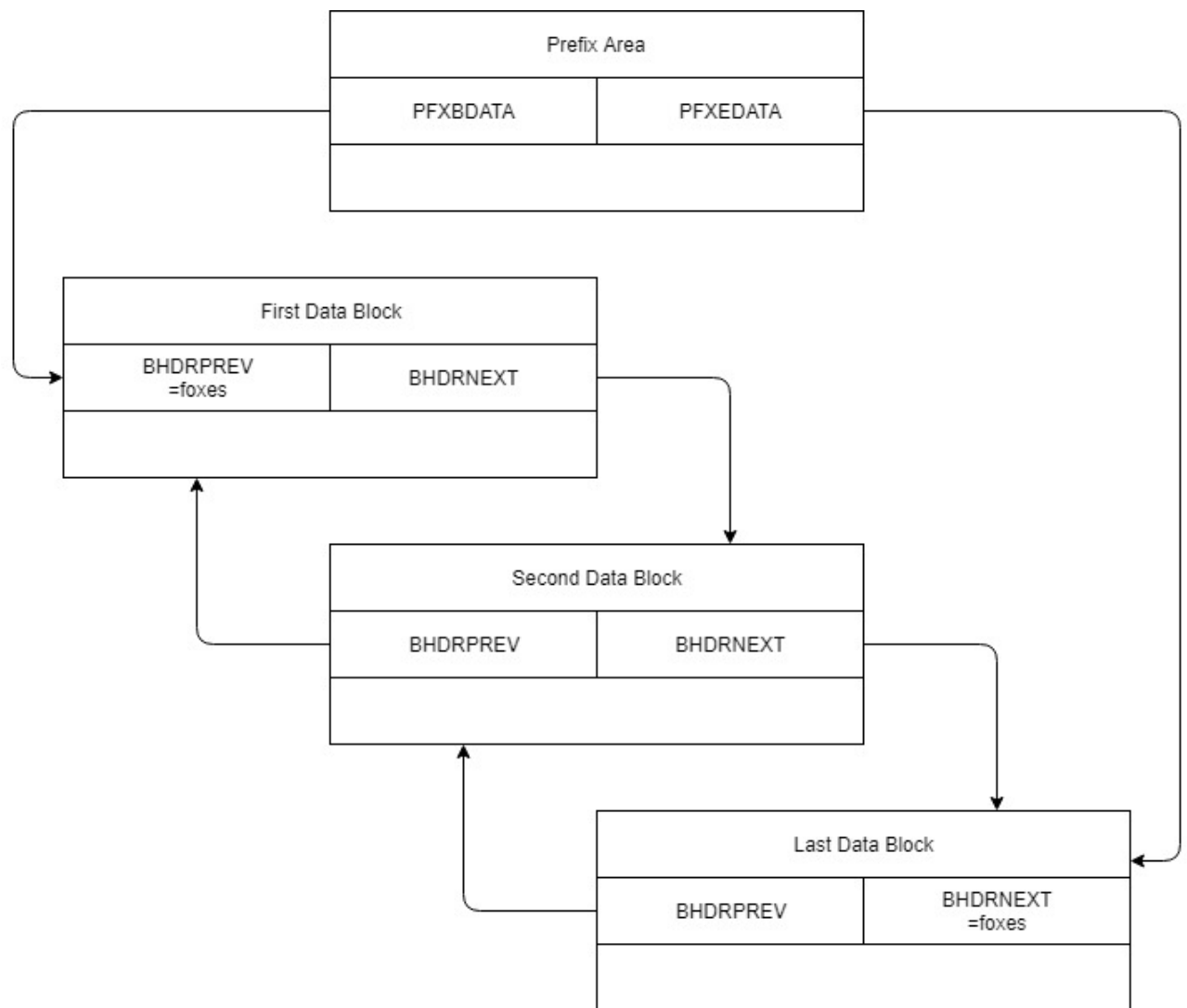
The first segment is on the Data chain and subsequent segments are retrieved via SPXBNEXT

The flag SPXSEGCC indicates the first, middle or last segments

Data Blocks

Data Block Structure (SPANNED=NO)

Assume we have a cluster with three data blocks holding records. The blocks are on the data chain as outlined in the picture below. Please note that all depicted s are block s. Each thus originates with the indicated field, and ends at the block it points to. The location where the arrows attach has no meaning since it's a block



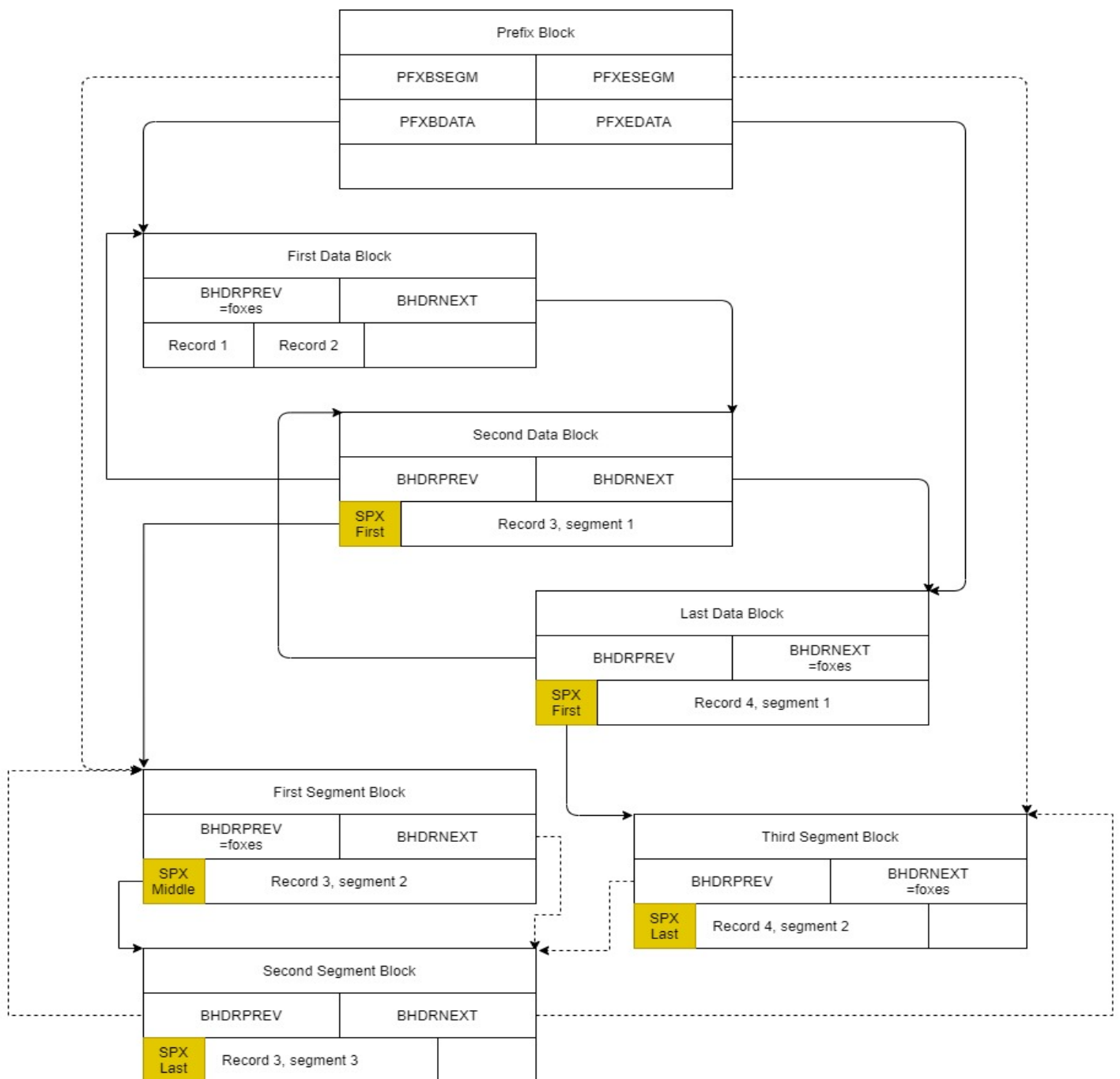
Data Block Structure (SPANNED=YES)

Now suppose we have a cluster with three data blocks, the first block holding two unsegmented records, the second block holding the first segment of a record consisting of three segments and the third block holding the first segment of a record consisting of two segments

In the picture we show the data chain as a solid line (as in the picture above), we show the segment chain as a dotted line, and we show the SPX s as a fat line

The picture shows the prefix area's to start/end block of both the data chain and the segment chain

It also shows the first and second block on each chain pointing to one another. Same thing for the second and third block on each chain



All depicted s are block s

Each originates with the indicated field, and ends at the block it points to

The location where the arrows attach has no meaning since it's a block

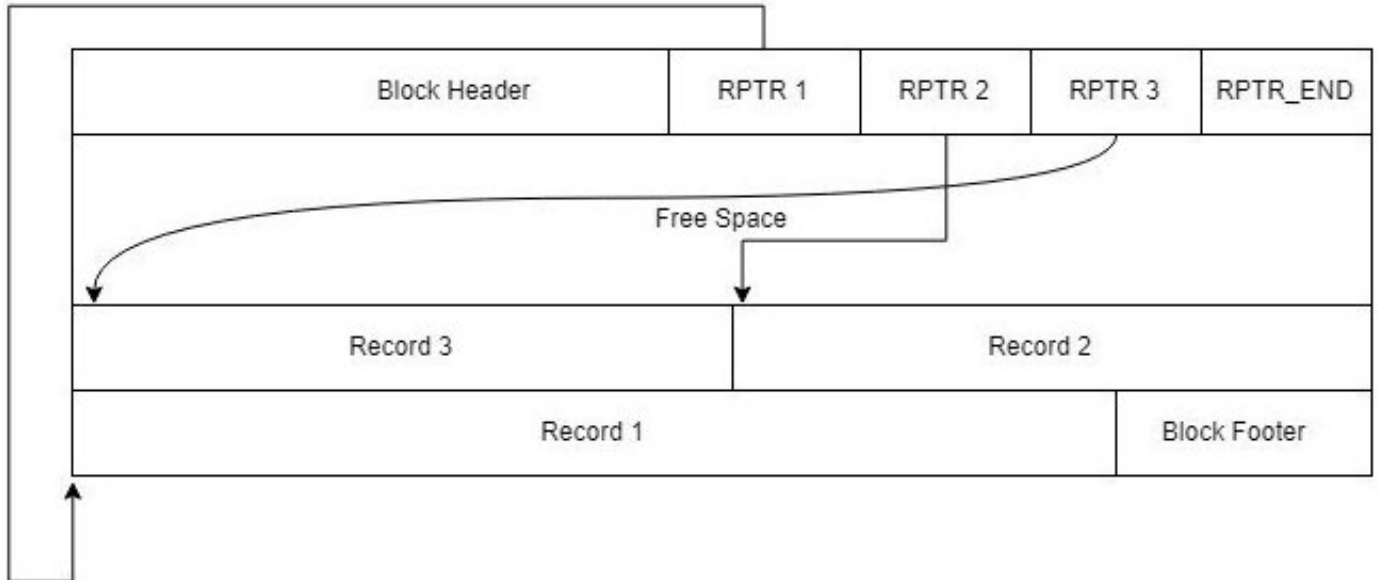
Data Block

Each record has an RPTR block, they are created after the Block Header

In addition to the offset, the RPTR contains flags to identify the type and status of each record

RPTR_END marks the end of records in this block

The records are placed in reverse order in the block to consolidate free space at the centre



It is possible to reserve an amount of freespace at load time which also applies if a block is split

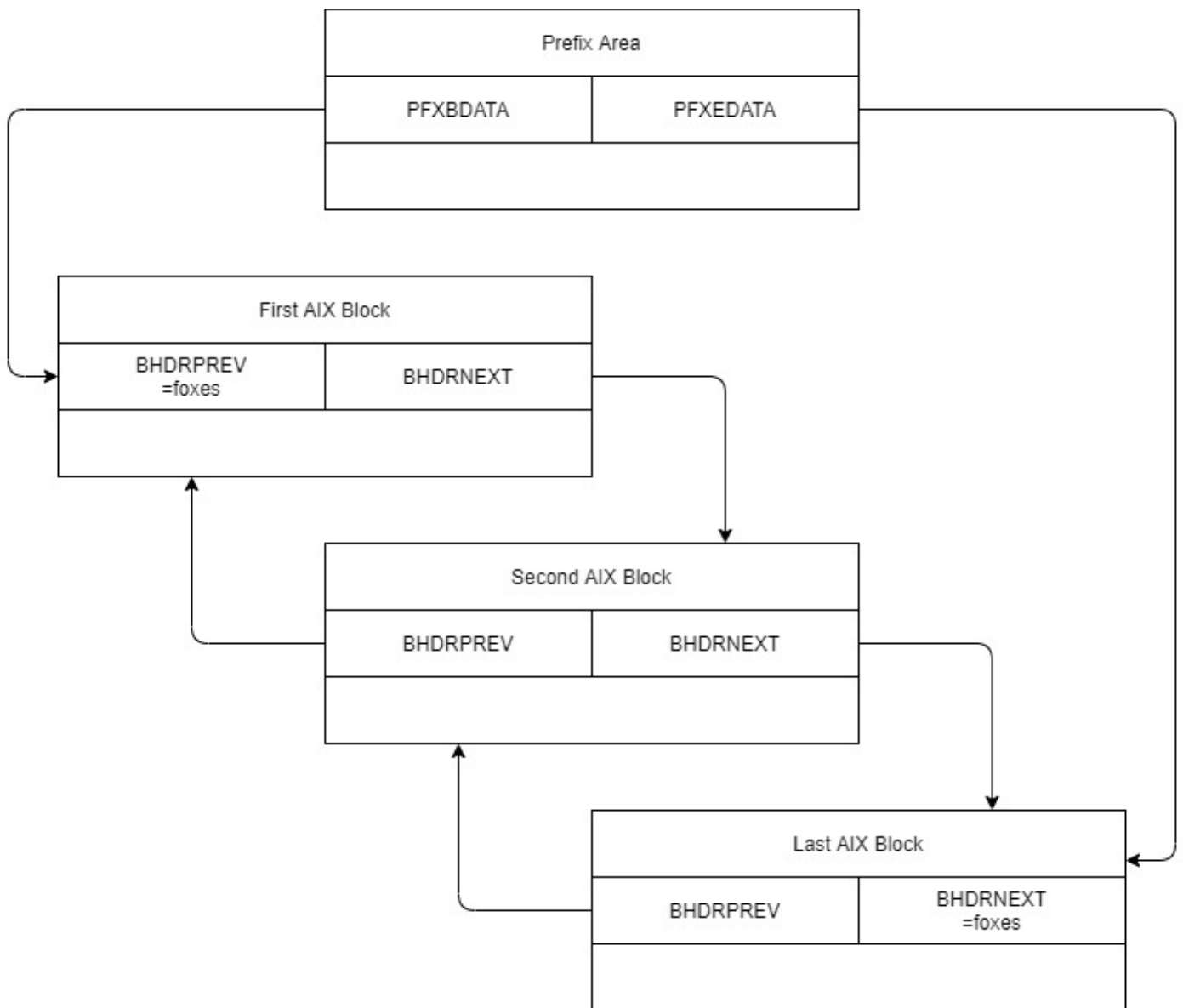
It is specified in the catalog as DATAFREESPACE=nn, where nn is a percentage of the available space

Only a fixed non-spanned KSDS can specify free space

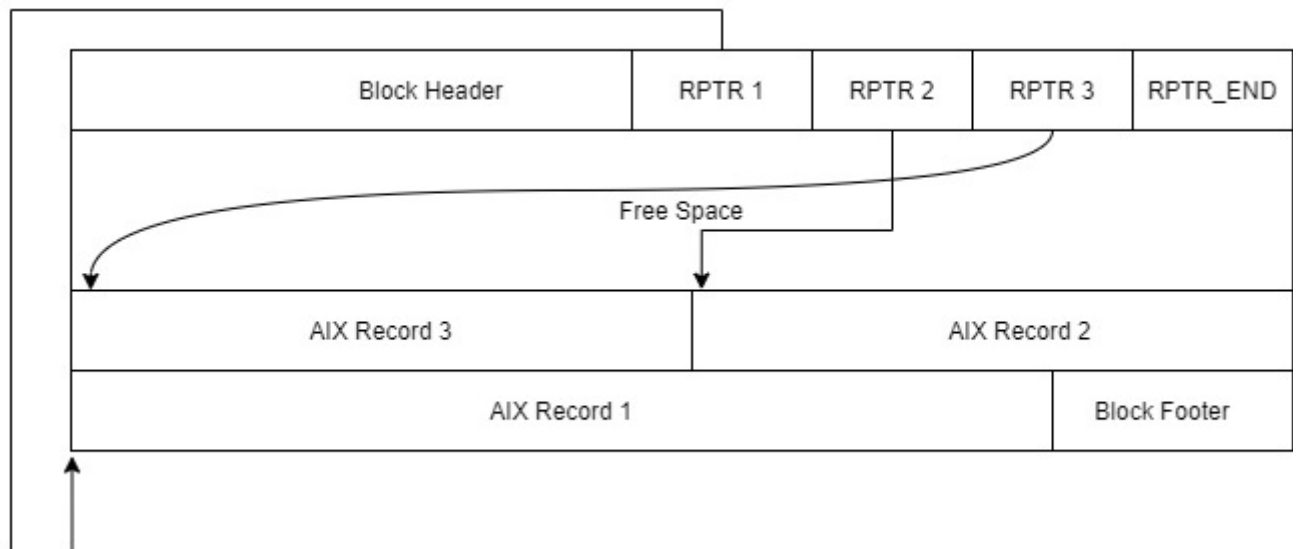
For all types of fixed non-spanned datasets, the available space may not be a multiple of the data record size resulting in unusable space. To correct this use DATAADJUST=YES which will calculate an optimal blocksize less than the specified one

AIX Blocks

AIX Block Structure (Unique)



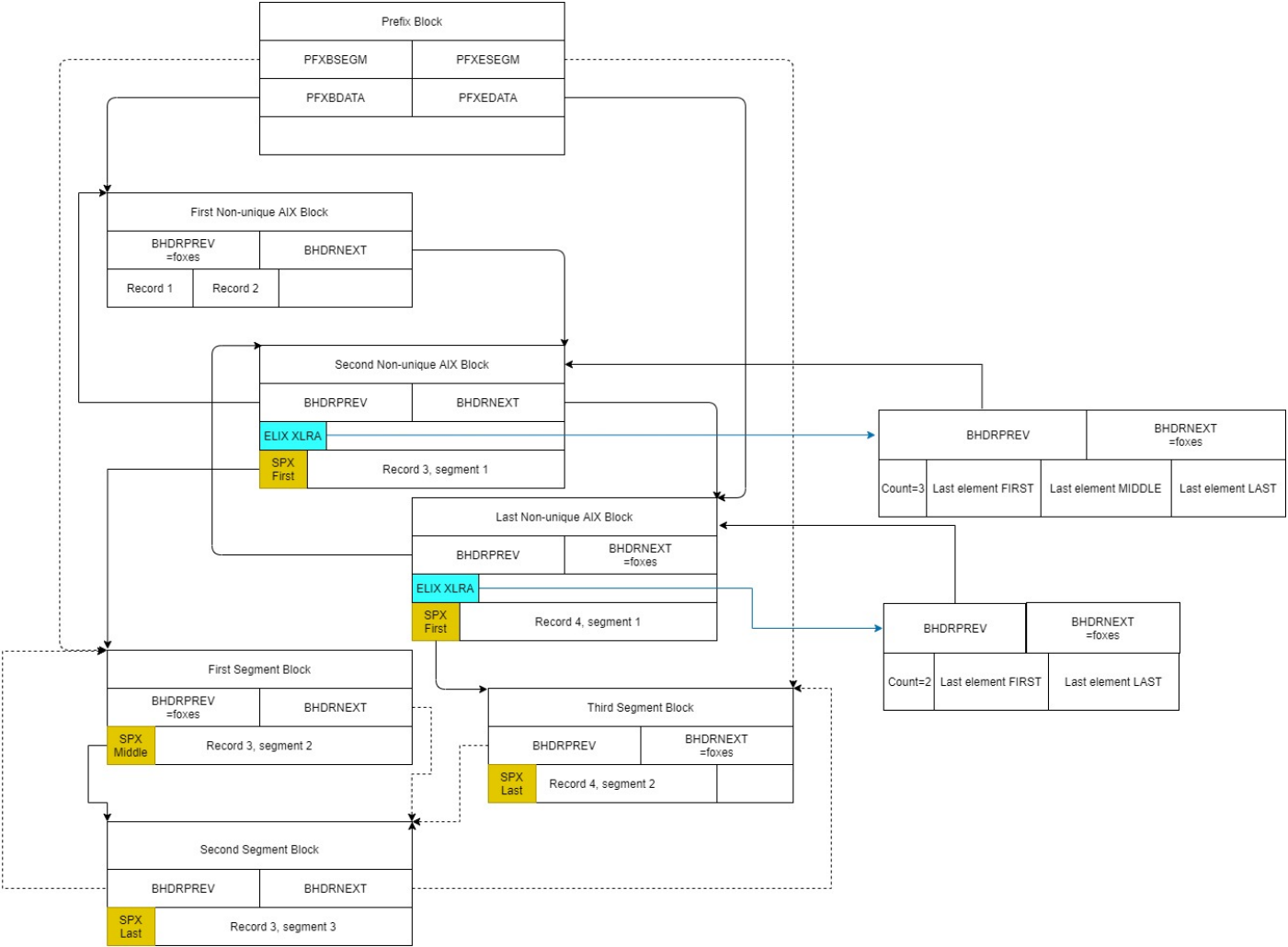
AIX Block (Unique)



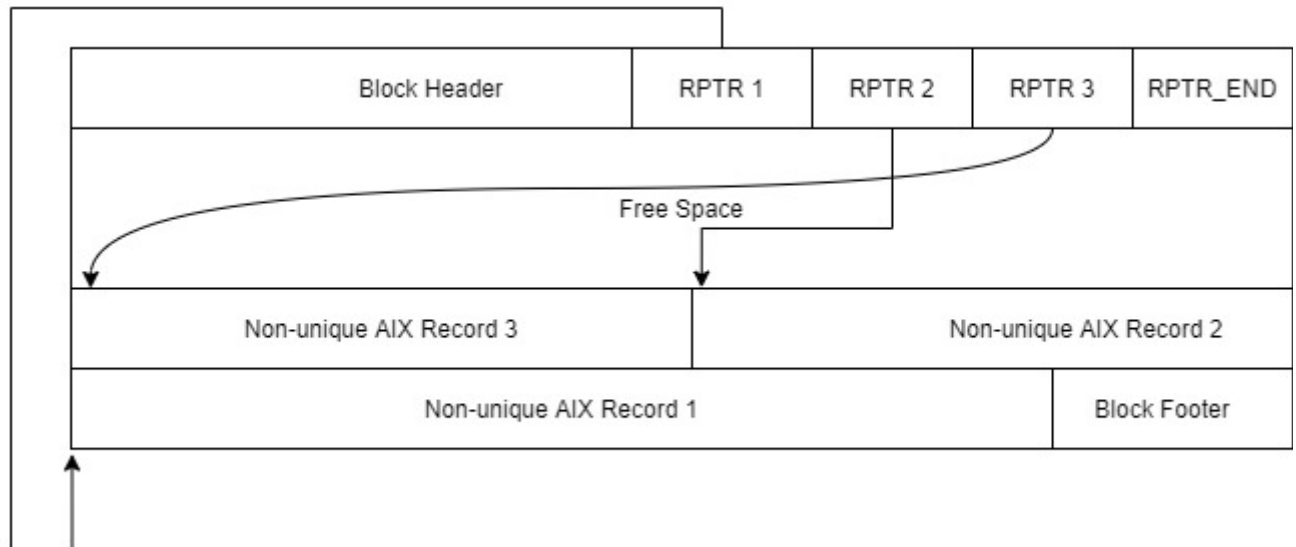
AIX unique records have the following format:

AIX on ...	Record Format
KSDS	AIX key followed by primary key
ESDS	AIX key followed by XRBA(8)

AIX Block Structure (Non-unique)



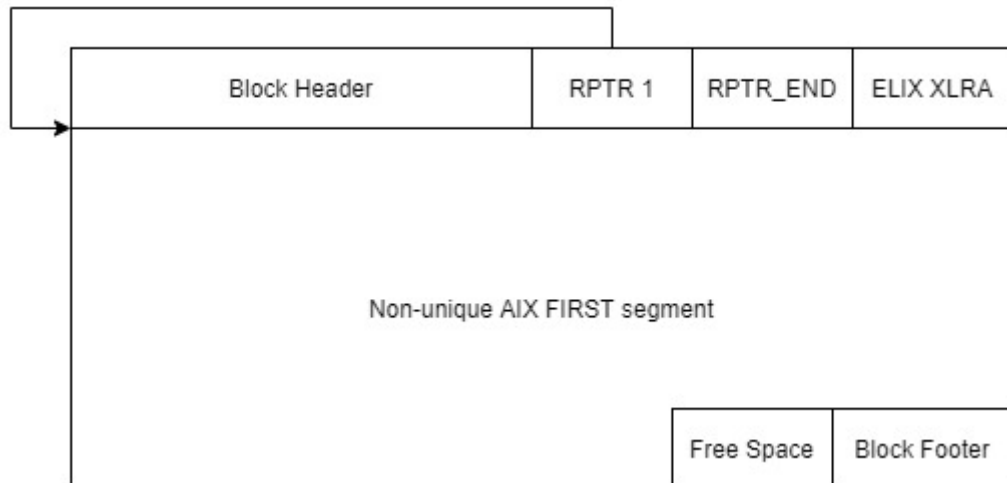
AIX Block (Non-unique and not segmented)



AIX non-unique non-segmented records have the following format:

AIX on ...	Record Format
KSDS	AIX key, an element count n(4) followed by n primary keys
ESDS	AIX key, an element count n(4) followed by n XRBAs(n*8)

AIX Block (Non-unique and segmented)



Each segment contains a whole number of elements

AIX non-unique segmented records have the following formats:

AIX on ...	Record Format of FIRST segment
KSDS	SPX, AIX key, an element count(4) which is the total no. of elements in all segments The actual number of primary keys in this segment can be calculated from SPXSEGLN
ESDS	SPX, AIX key, an element count(4) which is the total no. of elements in all segments The actual number of XLRA's in this segment can be calculated from SPXSEGLN

AIX on ...	Record Format of MIDDLE or LAST segments
KSDS	SPX and a number of primary keys The actual number of primary keys in this segment can be calculated from SPXSEGLN
ESDS	SPX and a number of XLRA's The actual number of XLRA's in this segment can be calculated from SPXSEGLN

ELIX Block

A single ELIX block is created for each non-unique AIX record that is segmented
It has the same blocksize as a Data record

zVSAM lifts the current IBM restriction of 32K elements in a non-unique AIX record, because of this there may be many segments to read to find an element to delete or an insertion point for a new record

The ELIX Block provides an extra index on the segments and contains the highest element in each segment
As there is currently only one ELIX Block per AIX key this places a limit on the number of elements

When a non-unique AIX is built zREPRO will issue a message on the log like this:

zREPRO AIX MAX ELEMENT LIMIT 87654

If the number of elements is too low then rebuild the AIX with a larger blocksize

IBM does not maintain elements in any particular order but for the ELIX structure to work zVSAM will maintain elements in sequence



The ELIX record has the following format:

AIX on ...	Record Format
KSDS	Highest Primary key followed by the XLRA of the segment (always record 1)
ESDS	Highest XRBA followed by the XLRA of the segment (always record 1)

Index Blocks

Each record has an RPTR block, they are created after the Block Header

In addition to the offset, the RPTR contains flags to identify the type and status of each record

RPTR_END marks the end of records in this block

The records are placed in reverse order in the block to consolidate free space at the centre

For Level 0 each record is the key (KSDS), XRBA (ESDS) or RRN (RRDS) and is followed by an XLRA

The XLRA is a record to the Data block

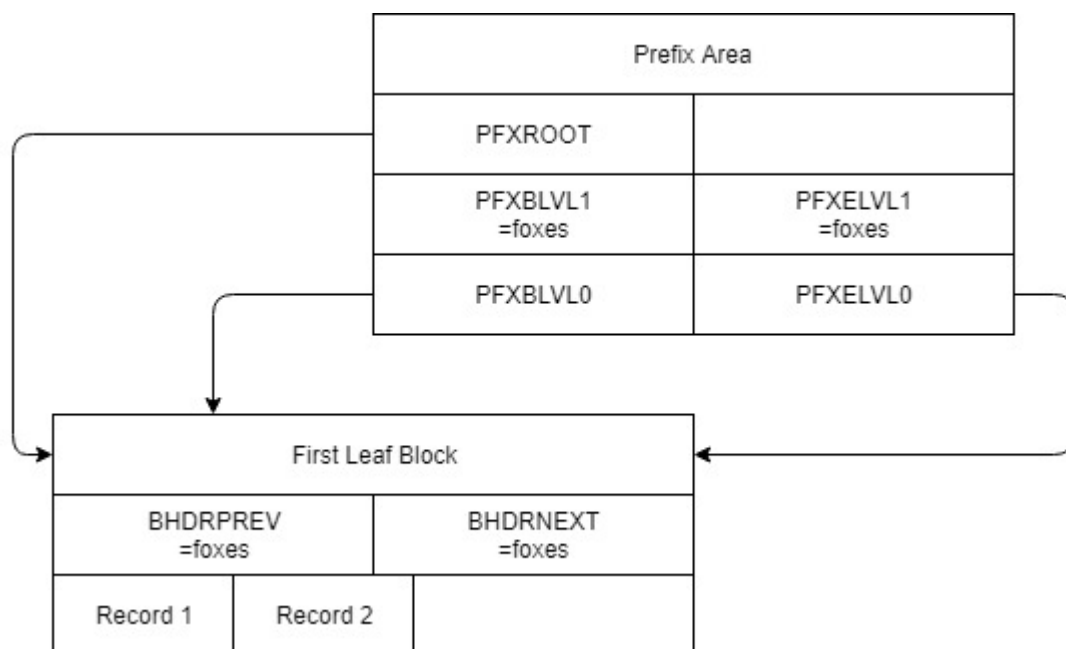
For other levels, each record is the highest key, XRBA or RRN followed by an XLRA

The XLRA is a block to the previous level

As each index record is a fixed size it is recommended to specify INDEXADJUST=YES to avoid unusable free space

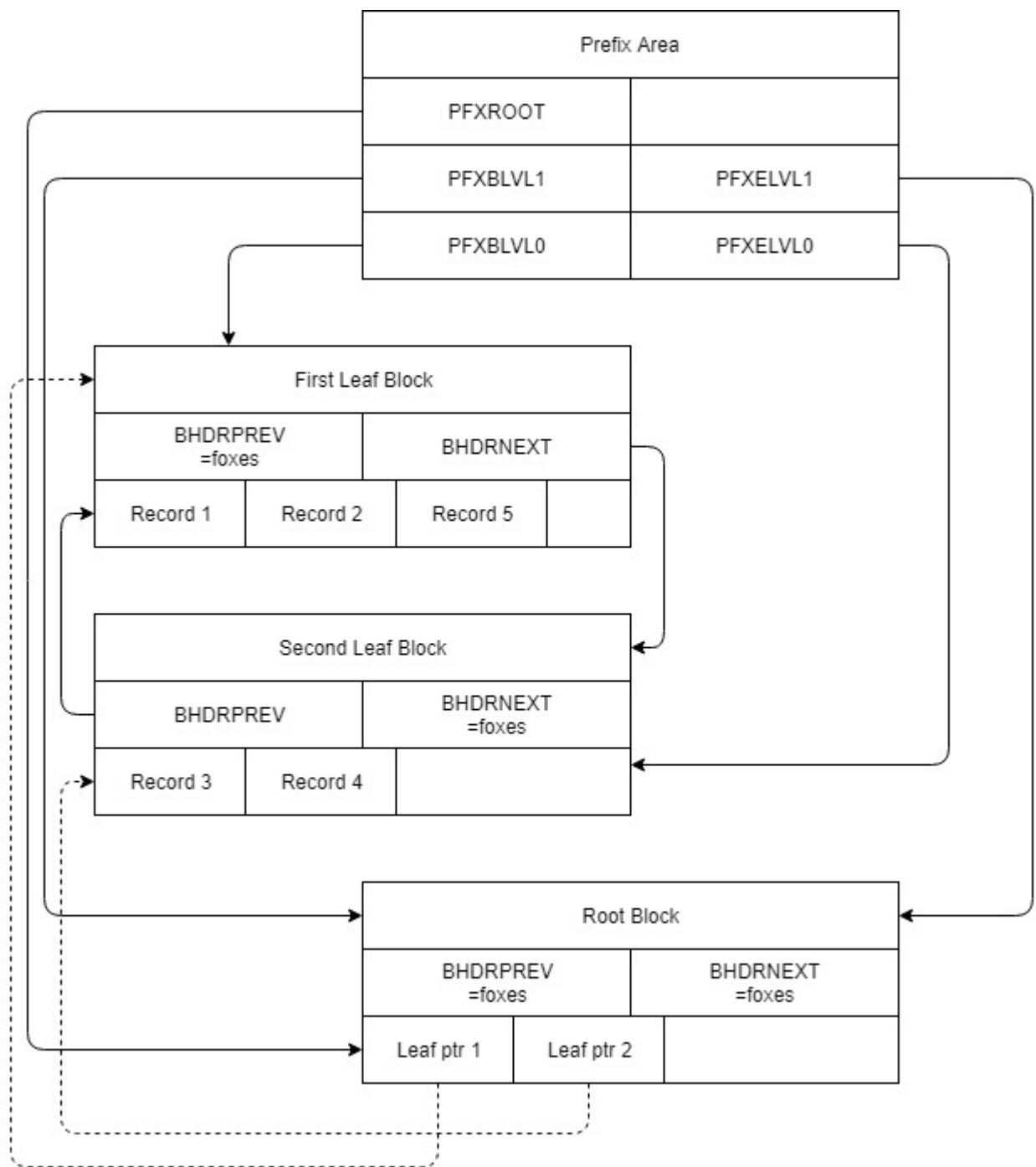
Index Block Structure: Single level

This example shows an index of only one block, holding two records

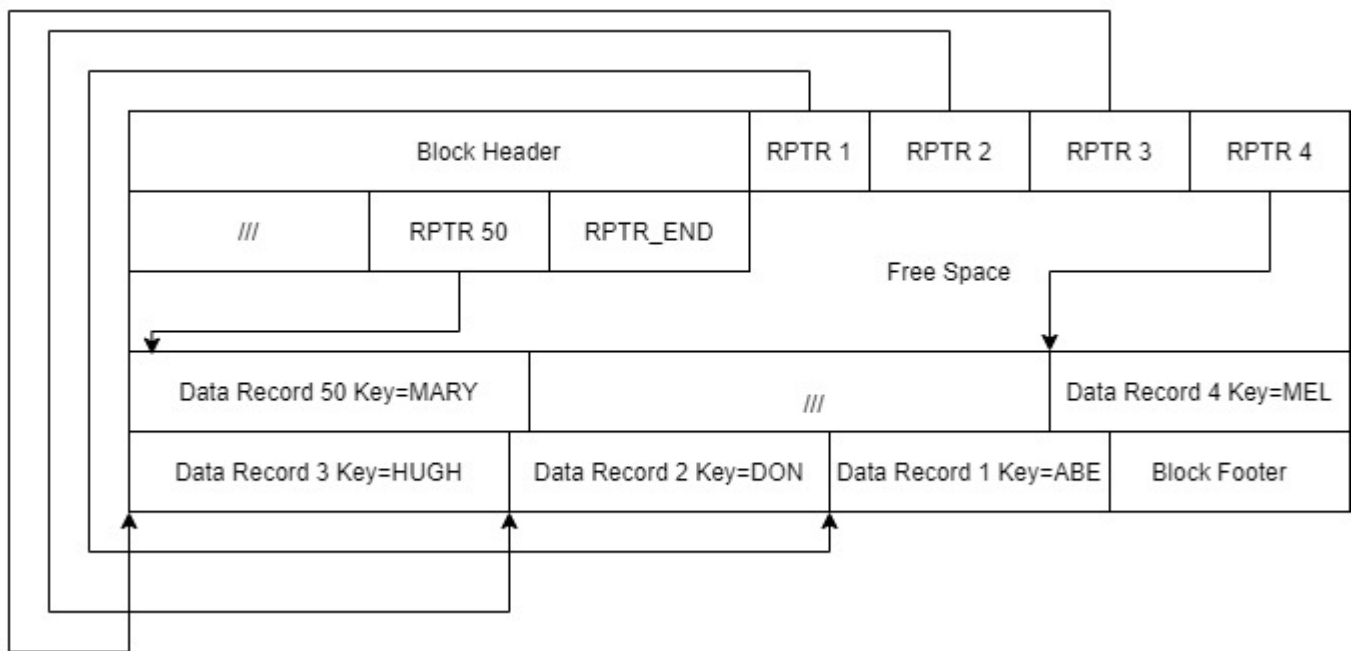


Index Block Structure: Two Levels

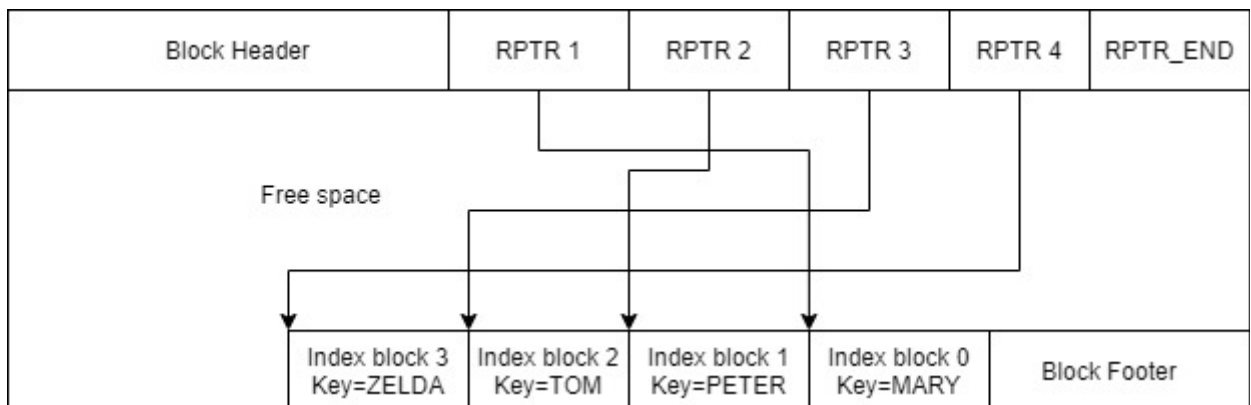
This example shows the index after adding three more records, causing the only index block to overflow and split. Now there are two leaf blocks, still on the LVL0 chain, and a new root block has been created on the LVL1 chain



Index Block Level 0



Index Block other levels



It is possible to reserve an amount of freespace at load time which also applies if a block is split
 It is specified in the catalog as INDEXFREESPACE=nn, where nn is a percentage of the available space
 Only a fixed non-spanned KSDS can specify free space

For all types of fixed non-spanned datasets, the available space may not be a multiple of the index record size resulting in unusable space. To correct this use INDEXADJUST=YES which will calculate an optimal blocksize less than the specified one

Structure and Functions by dataset type

KSDS Fixed non-Spanned

F-type records are conceptually stored one after another, filling the block until no space is left.

When the remaining free space is insufficient to accommodate another record, that free space remains unusable. Unusable space can be eliminated by building the dataset with DATAADJUST=YES

Blocks can be allocated with free space for adds (DATAFREESPACE=nn%), when the block is full the block will be split and any new block will have nn% free space

Format:

Block 1	Unusable Free Space	Record 3	Record 2	Record 1
---------	------------------------	----------	----------	----------

Function	Notes
Add	Yes
Update	Yes, the primary key must not be changed
Delete	Yes
Length change	n/a
Access by:	Primary key or AIX key (X)RBA not yet implemented

KSDS Fixed Spanned

FS-type records are conceptually stored one after another, using a block for each segment and starting each record on a new block. Record size is expected to exceed block size, so the record is split into segments, the first segment is created to fill an entire block, and the rest of the record goes into one or more secondary segments which are stored on the next blocks

Each segment is preceded by a Segment Prefix (SPX, marked in yellow)

zVSAM extension: The primary key and any AIX keys need not be in the first segment

Below we show an example where each record requires three segments:

Block 1	S P X	Record 1 - Segment 1 (FIRST)	
Block 2	S P X	Record 1 - Segment 2 (MIDDLE)	
Block 3	S P X	Record 1 - Segment 3 (LAST)	Unallocated
Block 4	S P X	Record 2 - Segment 1 (FIRST)	
Block 5	S P X	Record 2 - Segment 2 (MIDDLE)	
Block 6	S P X	Record 2 - Segment 3 (LAST)	Unallocated

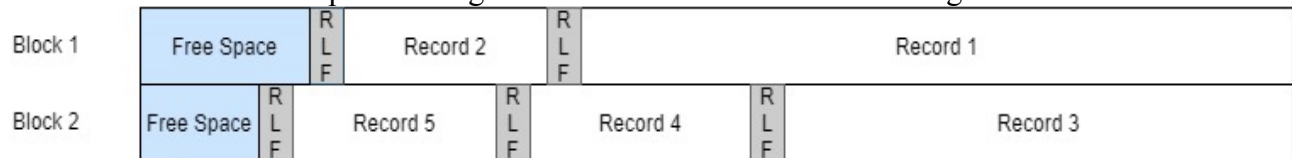
Function	Notes
Add	Yes
Update	Yes, the primary key must not be changed
Delete	Yes
Length change	n/a
Access by:	Primary key or AIX key (X)RBA not yet implemented

KSDS Variable non-Spanned

V-type records are conceptually stored one after another, filling the block until no space is left
Every record is preceded by a Record Length Field (RLF, marked in grey)

When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue) and the record is placed on the next block

Below we show an example showing how various numbers of records might fit into the blocks



Function	Notes
Add	Yes
Update	Yes, the primary key must not be changed
Delete	Yes
Length change	Yes When a record is shortened it must not affect the primary key or any AIX key
Access by:	Primary key or AIX key (X)RBA not yet implemented

KSDS Variable Spanned

VS-type records are conceptually stored one after another, filling the block until no space is left

Every record is preceded by a Record Length Field (RLF, marked in grey)

When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue) and the record is placed on the next block

Only if the record size exceeds the usable block size is the record split into segments and each segment is prefixed with a Segment Prefix. The first segment is created to fill an entire block, and the rest of the record goes into one or more secondary segments which are stored on the next blocks

Each segment is preceded by a Segment Prefix (SPX, marked in yellow)

zVSAM extension: The primary key and any AIX keys need not be in the first segment

Below we show an example showing how various numbers of records might fit into the blocks of the file, or how a single record might occupy multiple blocks of the file

Block 1	Free Space	R L F	Record 2	R L F	Record 1
Block 2	Free Space	R L F	Record 3		
Block 3	S P X	R L F	Record 4 - Segment 1 (FIRST)		
Block 4	Free Space		S P X	Record 4 - Segment 2 (LAST)	
Block 5	S P X	R L F	Record 5 - Segment 1 (FIRST)		
Block 6	S P X		Record 5 - Segment 2 (MIDDLE)		
Block 7	Free Space			S P X	Record 5 - Segment 3 (LAST)

Function	Notes
Add	Yes
Update	Yes, the primary key must not be changed
Delete	Yes
Length change	Yes When a record is shortened it must not affect the primary key or any AIX key
Access by:	Primary key or AIX key (X)RBA not yet implemented

ESDS Fixed non-Spanned

F-type records are conceptually stored one after another, filling the block until no space is left.

When the remaining free space is insufficient to accommodate another record, that free space remains unusable. Unusable space can be eliminated by building the dataset with DATAADJUST=YES

Format:

Block 1	Unusable Free Space	Record 3	Record 2	Record 1
---------	------------------------	----------	----------	----------

Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	No
Length change	n/a
Access by:	(X)RBA or AIX key

ESDS Fixed Spanned

FS-type records are conceptually stored one after another, using a block for each segment and starting each record on a new block. Record size is expected to exceed block size, so the record is split into segments, the first segment is created to fill an entire block, and the rest of the record goes into one or more secondary segments which are stored on the next blocks

Each segment is preceded by a Segment Prefix (SPX, marked in yellow)

zVSAM extension: Any AIX keys need not be in the first segment

Below we show an example where each record requires three segments:

Block 1	S P X	Record 1 - Segment 1 (FIRST)	
Block 2	S P X	Record 1 - Segment 2 (MIDDLE)	
Block 3	S P X	Record 1 - Segment 3 (LAST)	Unallocated
Block 4	S P X	Record 2 - Segment 1 (FIRST)	
Block 5	S P X	Record 2 - Segment 2 (MIDDLE)	
Block 6	S P X	Record 2 - Segment 3 (LAST)	Unallocated

Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	No
Length change	n/a
Access by:	(X)RBA or AIX key

ESDS Variable non-Spanned

V-type records are conceptually stored one after another, filling the block until no space is left
Every record is preceded by a Record Length Field (RLF, marked in grey)

When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue) and the record is placed on the next block

This dataset type is a zVSAM extension

Below we show an example showing how various numbers of records might fit into the blocks



Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	No
Length change	No
Access by:	(X)RBA or AIX key

ESDS Variable Spanned

VS-type records are conceptually stored one after another, filling the block until no space is left

Every record is preceded by a Record Length Field (RLF, marked in grey)

When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue) and the record is placed on the next block

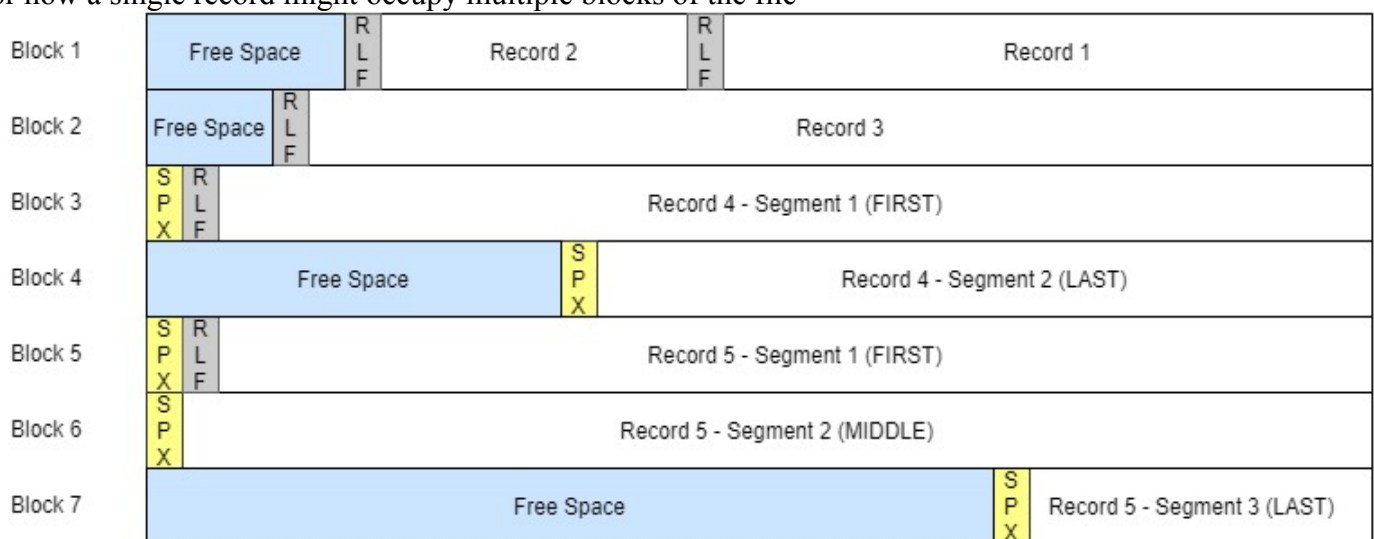
Only if the record size exceeds the usable block size is the record split into segments and each segment is prefixed with a Segment Prefix. The first segment is created to fill an entire block, and the rest of the record goes into one or more secondary segments which are stored on the next blocks

Each segment is preceded by a Segment Prefix (SPX, marked in yellow)

This dataset type is a zVSAM extension

zVSAM extension: Any AIX keys need not be in the first segment

Below we show an example showing how various numbers of records might fit into the blocks of the file, or how a single record might occupy multiple blocks of the file



Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	No
Length change	No
Access by:	(X)RBA or AIX key

RRDS Fixed non-Spanned

F-type records are conceptually stored one after another, filling the block until no space is left.

When the remaining free space is insufficient to accommodate another record, that free space remains unusable. Unusable space can be eliminated by building the dataset with DATAADJUST=YES

An RRDS consists of slots (RRNs) which may or may not contain a record

Empty slots are initially binary zeros with RPTR_MTY set

Block 1	Unusable Free Space	Slot 7 Record	Slot 6 Record	Slot 5 Record	Slot 4 (empty)	Slot 3 (empty)	Slot 2 (empty)	Slot 1 Record	Slot 0 Record
---------	------------------------	------------------	------------------	------------------	-------------------	-------------------	-------------------	------------------	------------------

Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	Yes, slots may not be deleted. RPTR_MTY is set
Length change	n/a
Access by:	RRN

RRDS Fixed Spanned

FS-type records are conceptually stored one after another, using a block for each segment and starting each record on a new block. Record size is expected to exceed block size, so the record is split into segments, the first segment is created to fill an entire block, and the rest of the record goes into one or more secondary segments which are stored on the next blocks

Each segment is preceded by a Segment Prefix (SPX, marked in yellow)

An RRDS consists of slots (RRNs) which may or may not contain a record

Empty slots are initially binary zeros with RPTR_MTY set

This dataset type is a zVSAM extension

Below we show an example where each record requires three segments:

Block 1	S P X	Slot 0 / Record - Segment 1 (FIRST)	
Block 2	S P X	Slot 0 / Record - Segment 2 (MIDDLE)	
Block 3	S P X	Slot 0 / Record - Segment 3 (LAST)	Unallocated
Block 4	S P X	Slot 1 / (empty) - Segment 1 (FIRST)	
Block 5	S P X	Slot 1 / (empty) - Segment 2 (MIDDLE)	
Block 6	S P X	Slot 1 / (empty) - Segment 3 (LAST)	Unallocated
Block 7	S P X	Slot 2 / Record - Segment 1 (FIRST)	
Block 8	S P X	Slot 2 / Record - Segment 2 (MIDDLE)	
Block 9	S P X	Slot 2 / Record - Segment 3 (LAST)	Unallocated

Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	Yes, slots may not be deleted. RPTR_MTY is set
Length change	n/a
Access by:	RRN

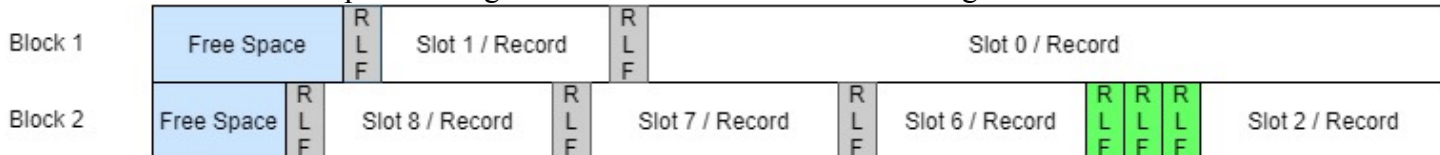
RRDS Variable non-Spanned

V-type records are conceptually stored one after another, filling the block until no space is left
Every record is preceded by a Record Length Field (RLF)

An RRDS consists of slots (RRNs) which may or may not contain a record
Empty slots consist of a dummy RLF containing X'00000004' with RPTR_MTY set, these are shown in green in the diagram. Non-empty slots have a grey RLF

When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue) and the record is placed on the next block

Below we show an example showing how various numbers of records might fit into the blocks



Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	Yes, slots may not be deleted. RPTR_MTY is set The record is replaced by a dummy RLF and the space is reclaimed
Length change	Yes
Access by:	RRN

RRDS Variable Spanned

VS-type records are conceptually stored one after another, filling the block until no space is left
Every record is preceded by a Record Length Field (RLF)

An RRDS consists of slots (RRNs) which may or may not contain a record

Empty slots consist of a dummy RLF containing X'00000004' with RPTR_MTY set, these are shown in green in the diagram. Non-empty slots have a grey RLF

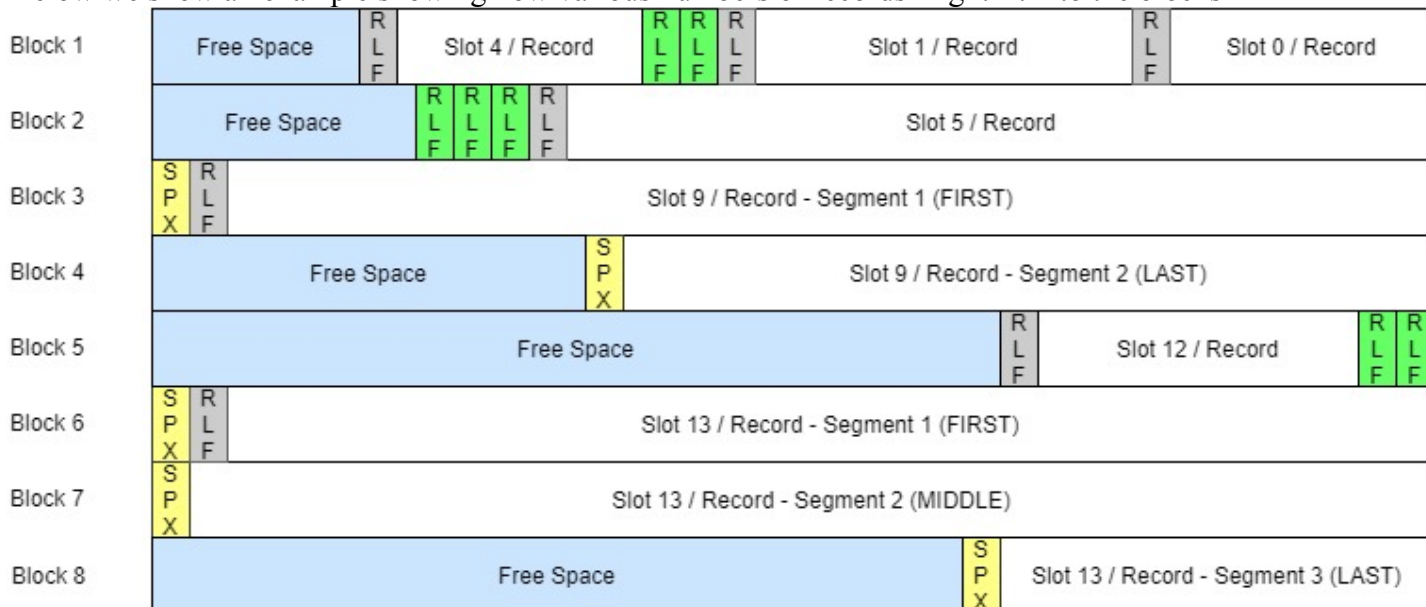
When remaining free space is insufficient to accommodate another record, that free space remains unallocated (marked in blue) and the record is placed on the next block

When a record length exceeds the available space in a block the record is split into segments, the first segment is created to fill an entire block, and the rest of the record goes into one or more secondary segments which are stored on the next blocks

Each segment is preceded by a Segment Prefix (SPX, marked in yellow)

This dataset type is a zVSAM extension

Below we show an example showing how various numbers of records might fit into the blocks



Function	Notes
Add	Yes, but only to the end of the dataset
Update	Yes
Delete	Yes, slots may not be deleted. RPTR_MTY is set The record is replaced by a dummy RLF and the space is reclaimed For segmented records, the freed blocks are marked as available
Length change	Yes
Access by:	RRN

Logical processes for RPL-based requests

POINT function

GET function

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer. If buffers need to be written out to make room for a block that needs to be read, then the CTRNEXCP counter needs to be incremented as well.

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field

If buffers need to be written out in order to allocate a buffer for a block that needs to be read, then prefix counter CTRNNUIW needs to be incremented

PUT function

Prefix counter field CTRAVSPAC must be updated. When consuming free space to allocate a new record, reduce CTRAVSPAC with space consumed. When allocating a new block (or splitting an existing block) add the blocksize – (block header, block footer, RPTR area) and reduce with amount used up (record length, including RDW/SPX and RPTR). When lengthening a record, subtract the difference; when shortening a record, add the difference

Prefix counter fields CTRHALCRBA and CTRENDRBA should be updated whenever a record is added beyond the current value of these fields. CTRENDRBA also should be updated in case the last record in the component is lengthened

Prefix counter field CTRNCIS should be incremented whenever a block needs to be split

Prefix counter field CTRNEXCP needs to be incremented whenever a block needs to be written out. This occurs when the cluster was opened with MACRF=NDF. With MACRF=DFR no writes are forced and no EXCP needs to be counted

Prefix counter fields CTRNINSR and CTRNLOGR need to be incremented whenever a new record is added to the component

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field. For basic put operation this is irrelevant, but for updating an index or an AIX reads may be needed and should be counted for the component being read

When an existing record is updated, then CTRNUPDR needs to be incremented

When adding a record the record length (including SPX/RDW, but excluding RPTR) needs to be added to the prefix counter field CTRSDTASZ. When a record is updated to a different length, then the difference in length needs to be accounted into the CTRSDTASZ field.

When a user write is forced then the prefix counter field CTRNUIW needs to be incremented. This happens when a put is issued to a cluster that was opened with MACRF=NDF. For clusters opened with MACRF=DFR writing is done by zVSAM when the buffer is needed for a different block. These writes are counted in the CTRNNUIW field

The prefix counter field CTRAVGRL contains the value of CTRSDTASZ / CTRNLOGR rounded up to the nearest integer. The field should be updated whenever either or both of the input values are changed

The prefix counter field CTRLOKEY@ is the offset to a length and a value of PFXYLEN bytes

The value should be updated whenever a record is inserted that has a lower key than the current lowest key.

ERASE function

Prefix counter field CTRNDELRL should be incremented for every successful erase operation

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer. If buffers need to be written out to make room for a block that needs to be read, then the CTRNEXCP counter needs to be incremented as well

Prefix counter field CTRNLOGR needs to be decremented for every successful erase operation

Prefix counter field CTRNRETR needs to be incremented for every block accessed, whether it needs to be read, or already resides in a buffer does not matter for this count field. For basic erase operation this is irrelevant, but for updating an index or an AIX reads may be needed and should be counted for the component being read

When erasing a record the record length (including SPX/RDW, but excluding RPTR) needs to be subtracted from the prefix counter field CTRSDTASZ

When a user write is forced then the prefix counter field CTRNUIW needs to be incremented. This happens when an erase is issued to a cluster that was opened with MACRF=NDF. For clusters opened with MACRF=DFR writing is done by zVSAM when the buffer is needed for a different block. These writes are counted in the CTRNNUIW field

The prefix counter field CTRAVGRL contains the value of CTRSDTASZ / CTRNLOGR rounded up to nearest integer. The field should be updated after every successful erase operation

The prefix counter field CTRLOKEY@ is the offset to a length and value of PFXKEYLN bytes

The value should be updated whenever the record is erased with the current lowest key

Prefix counter field CTRNEXCP needs to be incremented whenever a block is needed that does not yet reside in a buffer

CHECK function

ENDREQ function

VERIFY function

Locking

Addenda

API for RPL-based interfaces

POINT macro parameters

GET macro parameters

PUT macro parameters

ERASE macro parameters

CHECK macro parameters

ENDREQ macro parameters

VERIFY macro parameters

List of changes

Date	Author	Description
2018-09-16	Abe Kornelis	Remove SPX from VS records that have only a single segment. Change order and numbering of chapters Move macro parameter descriptions to addendum Expand chapter on compatibility In the addendum for GENCB ACB add explanation on MF usage
2018-09-18	Abe Kornelis	Various small changes as suggested by Hugh Sweeney Moved zACB and zEXLST layout paragraphs to the addenda.
2018-09-20	Abe Kornelis	Various small changes as suggested by Melvyn. See mail dated 2018-09-19 22:19
2018-09-27	Abe Kornelis	Added content for MODCB ACB, including addendum.
2018-09-29	Abe Kornelis	Added content for SHOWCB ACB, including addendum
2018-10-01	Abe Kornelis	Added content for TESTCB ACB, including addendum
2018-10-07	Abe Kornelis	Added comment on CBMR layout to chapters on GENCB ACB, MODCB ACB, SHOWCB ACB and TESTCB ACB. Parm AM=VSAM added to GENCB ACB chapter. Added content for GENCB EXLST, including addendum
2018-10-08	Abe Kornelis	Added content for MODCB EXLST, including addendum Added content for SHOWCB EXLST, including addendum Added content for TESTCB EXLST, including addendum
2018-10-09	Abe Kornelis	CBMR split into header and separate tail sections CBMR header description added
2018-10-10	Abe Kornelis	Minor changes as suggested by Melvyn's mail dd 2018-10-09 23:40 Addition of chapter titles for RPL-based interfaces to addenda.
2018-10-11	Abe Kornelis	Added CBMR description – body for ACB
2018-10-13	Abe Kornelis	Added CBMR description – body for EXLST Added RPL macro description, including addendum Added GENCB RPL macro description, including addendum Added MODCB RPL macro description, including addendum Added SHOWCB RPL macro description, including addendum Added TESTCB RPL macro description, including addendum
2018-10-15	Abe Kornelis	Added ACBPFX to zACB layout
2018-10-16	Abe Kornelis	Added CBMR description – body for RPL ACBTYPE → ACBSTYPE Removed ACBMACR3_NLW and ACBMACR3_MODE ACBCUEL → ACBUEL ACBOCK → ACBLOCK
2018-10-21	Abe Kornelis	ACB ADR/KEY improved keyword description in addendum ACB IN/OUT improved keyword description in addendum ACB DDNAME improved keyword description in ACB macro chapter and the addendum Unsupported parameters and keywords on ACB, EXLST, RPL changed from “flagged as error” to “ignored”
2018-10-22	Abe Kornelis	Add description of prefix block, including counters area.

Date	Author	Description
		Updated addendum for SHOWCB/TESTCB with reference to source of data for each keyword. Added prefix field PFXIXLVL. Added instructions for RPL-based operations on how to maintain prefix counter fields. Added description of spacemap block.
2018-10-23	Abe Kornelis	Specify that SHOWCB/TESTCB for RBA/XRBA is supported for ESDS only. Foxes for any other component.
2018-10-24	Abe Kornelis	Add description for block header, block, footer, record list
2018-10-26	Abe Kornelis	Added description of open macro logic
2018-10-27	Abe Kornelis	Added eyecatcher to the prefix area, moved record length and key info fields to beginning of prefix area BHDRPREV/NEXT on prefix block documented as being foxes
2018-10-28	Abe Kornelis	Added ACBVER to zACB layout Added Area to Terminology chapter Max. block size reduced from 2G to 16MB Added ACBXPFX to zACB layout Added description of open execution logic
2018-11-21	Abe Kornelis	In API on macro interfaces improved wording for handling of (as yet) unsupported macro parameters. Add ATRB=VESDS for TESTCB ACB Improved picture and text on spacemap block layout
2018-11-22	Abe Kornelis	BHDRPREV/NEXT details expanded Removed PFXBSEG/PFXESEG
2018-11-25	Abe Kornelis	Spacemap area structure. Segmented records were missing. Added.
2018-11-29	Abe Kornelis	Added diagrams to chapter on block header structure.
2018-12-02	Abe Kornelis	Added alternative diagram for chaining segments. Preferred solution not yet determined And added drawings for chaining index blocks
2018-12-09	Abe Kornelis	Structure of Physical files: ELIX added to the list of block types Block Header Structure: BHDRFLGS changed to BHDRFLG1 and added BHDRFLG2 with BHDR_ELX
2018-12-17	Abe Kornelis	Put PFXBSEG/PFXESEG back in Corrected typos in drawings for explaining BHDRNEXT/PREV RPTR_END no longer all foxes, foxes only for RPTRREC@ Added 4 date fields to the prefix structure for creation and last update timestamps for both data and index component. MF=omitted changed to MF= in various locations
2019-01-06	Abe Kornelis	Added various fields to RPL

Date	Author	Description
2019-01-23	Melvyn Maltz	Version 2.0 Took over the D&L, not documenting spelling, syntax, bad references and other trivia Added hyperlinks Amended CBMRACB fields Amended CBMRRPL fields Removed RPL TIMEOUT...VTAM only Updated Prefix Block DSECT
2019-02-17	Melvyn Maltz	Version 2.1 Corrected diagram "Segmented Data Block Structure" CBMR RPL – Added GEN MOD SHOW TEST column CBMR ACB – Added GEN MOD SHOW TEST column CBMRACB RMODE31 – Added description
2019-02-21	Melvyn Maltz	Corrections to RPL DSECT. Added RPLFEEDB Note added to AIXPC
2019-02-22	Melvyn Maltz	Corrections and updates to ACB DSECT
2019-02-25	Melvyn Maltz	Added note to MODCB ACB about turning off MACRF=OUT
2019-02-26	Melvyn Maltz	Revised the OPEN Execution Logic table to distinguish V1 from V2
2019-03-02	Melvyn Maltz	Revised the EXLST DSECT
2019-03-03	Melvyn Maltz	Added section EXLST Macro Logic CBMR EXLST – Added GEN MOD SHOW TEST column Added _MODS in preparation for MODCB
2019-03-10	Melvyn Maltz	Removed sections on Logical Processes that don't involve calls to Java eg. MODCB. These are already well described
2019-03-12	Melvyn Maltz	Added Return and Reason Codes to MODCB ACB, RPL and EXLST
2019-03-17	Melvyn Maltz	Revised all sections on OPEN and CLOSE Added "CLOSE Macro Logic" Removed OPCL DSECT, replaced with list formats
2019-03-18	Melvyn Maltz	Added Data Block diagram Most references and diagrams for LDS removed, too long term
2019-04-15	Melvyn Maltz	Added hyperlinks to EXLST CBMR Modifiers
2019-04-17	Melvyn Maltz	Added UPAD= and RLSWAIT= Although not supported, they need to exist Amended the EXLST CBMR keyword values to fit them in Added WAREA, LENGTH and LOC to EXLST CBMR
2019-05-19	Melvyn Maltz	Added WAREA, LENGTH and LOC to ACB CBMR Amended the ACB CBMR keyword values to fit them in
2019-06-26	Melvyn Maltz	Added WAREA, LENGTH and LOC to RPL CBMR Added Return and Reason Codes to GENCB ACB, RPL and EXLST
2019-06-29	Melvyn Maltz	Changed PFXMAPOF from 3 to 4 bytes, adjusted following offsets

Date	Author	Description
2019-06-30	Melvyn Maltz	Version 2.2 Open Execution Logic, removed references to OC24/OC31/OCPL
2019-07-06	Melvyn Maltz	RPLACB changed to RPLDACB to match IBM
2019-07-07	Melvyn Maltz	CBMR EXLST: Added CBMRXL_AREA and _EXLST Marked fields used for SHOWCB
2019-07-08	Melvyn Maltz	zVSAM V2 compatibility with zVSAM V1 Added item 4 about re-assembling modules with OPEN
2019-08-17	Melvyn Maltz	EXLST Macro corrected Extra comments added to clarify missing parms for GENCB
2019-08-24	Melvyn Maltz	AM=VSAM added to Macros SHOWCB EXLST, UPAD and RLSWAIT removed, IBM doesn't support them. JRNAD will return zero
2019-10-09	Melvyn Maltz	Added subfields to RPLFEEDB Added RPLCXRBA
2019-10-13	Melvyn Maltz	Added CBMRACB_ACB and CBMRRPL_RPL for SHOWCB Renamed CBMRRPL_CRBA to CBMRRPL_RBA
2019-10-20	Melvyn Maltz	Added CBMRRPL_RECAREA to resolve conflict between SHOWCB RPL AREA= and FIELDS=AREA Renumbered CBMRRPL_RPLLEN to X'6E' to avoid conflict with CBMRXL_XLSTLEN Added CBMRRPL_TRANSID for SHOWCB
2019-10-27	Melvyn Maltz	Corrected syntax to all forms of MF=L SHOWCB, ACB, EXLST and RPL have all 3 lengths: ACBLEN, EXLLEN and RPLLEN New section added "No block type specified" and SHOWCB for extracting lengths only New section added "CBMR description-body for no block specified"
2019-11-13	Melvyn Maltz	SHOWCB ACB FIELDS revised Added several X~ fields as SHOWCB counters expect 4 bytes and the CTR fields are 8 bytes
2020-01-26	Melvyn Maltz	SHOWCB ACB revisions CBMRACB_RMODE31 and CBMRACB_SHRPL codes changed to avoid conflict with CBMRRPL_RPLLEN and CBMRXL_XLSTLEN
2020-02-24	Melvyn Maltz	CBMRACB_SDTASZ added as an 8-byte field CTRSDTA renamed to CTRSDTASZ References to a value of zero returned removed
2020-03-01	Melvyn Maltz	Corrections made to the description of CTRLOKEY@ This is an offset to LL+key
2020-03-02	Melvyn Maltz	Added CBMRACB_AREA

Date	Author	Description
2020-03-15	Melvyn Maltz	Added CBMRACB_XNINSR
2020-04-28	Melvyn Maltz	Reconstructed the Macro description sections to be more like that of the VSAM Macro manual Separate chapter for the xCB Macro MF= with hyperlinks to it from each of them, duplicate MF= descriptions removed
2020-05-28	Melvyn Maltz	Added diagrams for Data and Index (L0 and Ln) blocks
2020-05-29	Melvyn Maltz	Version 2.3 Removing individual sections on Fixed, Variable etc. and ESDS, KSDS and RRDS Removing Concepts Replaced with all 12 dataset type diagrams with descriptions Removed Displaced Record structure Added SPX format Added AIX structures and formats Added ELIX structures and formats Removed Counters area values, these are referenced in the macros themselves Added Implied OPEN table Added Close execution logic (incomplete) All OPEN-related, EXLST/Exit-related and CLOSE-related doc now in single chapters Addenda items moved to the appropriate chapter or deleted Deleted all DSECTs, too difficult to maintain both the DSECT and the doc The DSECTs are fully commented and offsets can be seen in an assembly CBMR details moved to the xCB chapter and simplified
2020-08-18	Melvyn Maltz	Version 2.4 Added paragraph in MODCB RPL= to clarify the OPTCD= parameter
2020-08-22	Melvyn Maltz	Revised the xCB return codes Revised the paragraph in MODCB ACB= about the MACRF= parameter
2020-09-25	Melvyn Maltz	Improved the wording of MF= for the xCB macros Clarified what is returned in R0 and R1 for the xCB macros
2020-10-10	Melvyn Maltz	RPL and MODCB have notes about internal/external ECBs and the bit RPLOPT2_ECB
2020-10-19	Melvyn Maltz	Updated OPEN execution logic to include fields and bits in the ACB which must be set on a successful OPEN
2020-11-08	Melvyn Maltz	Added a paragraph about syntax checking in API for Assembler...
2020-11-23	Melvyn Maltz	Added section “GENCB, MODCB, TESTCB and SHOWCB parameter types”

Date	Author	Description
2020-12-01	Melvyn Maltz	Revised all sections for TESTCB Removed Addenda, chapter on “API for ACB-based interfaces” Info is incorporated into the macro description CBMR verb codes amended to X'01'-X'DF' as 4-byte fields So few 0-byte fields that it wasn't worthwhile having a separate category
2020-12-17	Melvyn Maltz	SHOWCB LOKEY, added comment and hyperlink to errors
2021-01-25	Melvyn Maltz	Note added in Terminology to clarify the meaning of XRBA
2021-05-11	Melvyn Maltz	Revised OPEN and CLOSE sections to include the zOPC header, storage violation attempt and V1 vs V2 detection
2021-05-13	Melvyn Maltz	Revised OPEN and CLOSE sections to include the MODE bit as the senior bit of no. of entries, code to prevent MF=E modifying an MF=L with a different mode
2021-06-19	Melvyn Maltz	Added new section “Counters Area and its maintenance “